Welcome

Hi, I'm Mason Egger, Senior Technical Curriculum Developer at Temporal Technologies. I want to welcome you to Temporal 101, where you will learn and use the basic building blocks that enable you to develop durable applications on the Temporal platform.

This course is intended for experienced developers, although it requires no prior knowledge of Temporal. The code used in examples, including the code you'll work with during hands-on exercises, is written in the Python programming language, so you'll need at least a basic understanding of its syntax.

Thank you for your participation and I hope you enjoy the course.

Format and Duration

Temporal 101 teaches the fundamentals of Temporal development through a combination of text and video explanations, diagrams, demonstrations, and hands-on exercises. It also provides links to other resources of interest, including relevant documentation and sample code.

This is a self-paced course, so you can progress through it as slowly or quickly as you like. We find that the typical student takes around two hours to complete it. We recommend that you complete each of the recommended activities in the order presented, starting with the first lesson. However, you can see your progress at the top of the screen and jump to any of the sections of the course.

Course Outcomes

In this course, you will explore the basic building blocks of Temporal: Workflows and Activities. You'll use these building blocks along with Temporal's Python SDK to develop a small application that communicates with an external service. You'll see how Temporal helps you recover from failures and explore Temporal's execution model and event history. You'll use the Temporal Web UI and Temporal's command-line tools to explore and interact with your Workflows, and you'll use what you've learned to add new features to your existing workflow.

After completing this course, you will be able to:

⚙️ Configure Your Environment

- Configure an environment for developing Temporal Applications
- Find Temporal's documentation and code samples
- Deploy a Temporal cluster for development use
- Install a Temporal SDK

- Install the `temporal` CLI
- Understand the different options for running Temporal in production

🔁 Build Basic Workflows

- Use Temporal to describe and implement a basic business process
- Write a Workflow Definition
- Pass parameters into a Workflow
- Access the result of a Workflow
- Pass parameters into an Activity
- Get the result of an Activity from within a Workflow
- Interface with an external service from an Activity

🧠 Understand the Execution Model

- Interpret the Workflow Execution model
- Understand the relationship between a Worker and the Temporal Server
- Understand deployment and connectivity requirements for an application
- Locate and interpret the Event History (and input/output values) for a Workflow Execution

🖥️ Use the Web UI & CLI

- View current status of a Workflow (from Web UI)
- View history of past Workflows (from Web UI)
- View history of past Workflows (from command line)

🚀 Manage Your Application Lifecycle

- Execute a Workflow (from the command line)
- Execute a Workflow (from code)
- Make minor changes to your application code
- Restart the Worker process following deployment of new code

---

Setting up a local development environment

(Atividade opcional) Configurando um ambiente de desenvolvimento local

Se você preferir trabalhar neste curso no seu próprio computador em vez de usar o ambiente online, pode configurar um ambiente de desenvolvimento por conta própria.

Nesta lição, você instalará as ferramentas e arquivos necessários e iniciará um cluster Temporal local para executar seus Workflows e Activities.

Baixe os softwares necessários

Para concluir este curso, você precisará dos seguintes itens instalados no seu dispositivo:

- **Temporal CLI** — Siga as instruções de Set up a local Temporal development cluster para instalar o Temporal no seu dispositivo.
- **Git** — Se não tiver o Git instalado, siga as instruções de instalação na documentação oficial.
- **Python 3.10** — Este curso utiliza a versão 3.10 da linguagem.
- **Editor de texto** — O Visual Studio Code é uma boa opção, mas qualquer editor com suporte a Python funcionará.

Com esses programas instalados, você pode baixar os arquivos do curso.

Baixe os exercícios e exemplos do curso

Os arquivos deste curso estão no repositório de código no GitHub.

Clone o repositório:

```
git clone https://github.com/temporalio/edu-101-python-code
```

Acesse o diretório clonado:

```
cd edu-101-python-code
```

Crie um ambiente virtual:

Windows:

```
python -m venv env
```

Linux e/ou macOS:

```
python3 -m venv env
```

Ative o ambiente virtual:

Windows:

```
env\Scripts\activate
```

Linux e/ou macOS:

```
source env/bin/activate
```

Instale as dependências:

```
python -m pip install -r requirements.txt
```

Inicie o Cluster Temporal

Em um terminal separado, execute o seguinte comando:

```
temporal server start-dev --ui-port 8080
```

O servidor será iniciado. Aguarde todos os serviços subirem e acesse `localhost:8080` para visualizar o Temporal Web UI.

Abra outra janela de terminal

O Cluster Temporal precisa permanecer em execução durante todo o curso, por isso você precisará de outra janela de terminal aberta.

Acesse a pasta do repositório clonado:

```
cd edu-101-python-code
```

> Certifique-se de ativar o ambiente virtual Python em cada novo terminal que abrir.

Você usará esta janela de terminal para executar os comandos do curso.

Seu ambiente local está configurado. Continue para a próxima lição.

Convenções do Curso

Capitalização

Você pode notar que certos termos usados no curso, como Workflow, Activity e Event History, aparecem com letra maiúscula. Essa é uma convenção adotada no Temporal para distinguir entre um conceito geral (escrito em minúsculas) e uma implementação específica do Temporal (escrita com maiúscula).

Por exemplo, usamos "workflow" para descrever um processo de negócio em geral, mas usamos "Workflow" para descrever uma implementação desse processo utilizando as APIs do Temporal.

Código dos Exercícios

Cada exercício do curso estará contido em um subdiretório específico dentro do diretório `exercises` no ambiente do curso. Esse subdiretório conterá duas pastas:

- `practice` — onde você fará as alterações descritas nas instruções do exercício.
- `solution` — contém o exemplo completo e funcional, que você pode usar para conferir seu trabalho ou como dica caso precise.

Sempre que possível, o código na pasta `practice` utiliza comentários iniciados com `TODO` para indicar as partes do código que precisam ser modificadas.

Transcrições dos Vídeos

Fornecemos uma transcrição para cada vídeo do curso. Elas estão localizadas abaixo do título **"Video Transcript"** na página onde o vídeo está incorporado.

Objetivos do Capítulo 2

O que esperar

Neste capítulo, você trabalhará com o objetivo de ser capaz de configurar um ambiente para desenvolver aplicações Temporal.

Especificamente, ao final deste capítulo, você será capaz de:

- Encontrar a documentação e os exemplos de código do Temporal
- Executar um Cluster Temporal para uso em desenvolvimento
- Instalar um SDK do Temporal
- Instalar o `temporal` CLI
- Entender as diferentes opções para executar o Temporal em produção

---

Introdução ao Temporal

O que é o Temporal?

Em resumo, o Temporal é uma plataforma que garante a execução durável do código da sua aplicação. Ele permite que você desenvolva como se falhas simplesmente não existissem. Sua aplicação funcionará de forma confiável mesmo que encontre problemas como quedas de rede ou travamentos de servidor, que seriam catastróficos para uma aplicação comum. A plataforma Temporal lida com esses tipos de problemas, permitindo que você se concentre na lógica de negócio em vez de escrever código para detectar e se recuperar de falhas.

Workflows

As aplicações Temporal são construídas usando uma abstração chamada Workflows. Você desenvolverá esses Workflows escrevendo código em uma linguagem de programação de uso geral, como Go, Java, TypeScript ou Python. O código que você escreve é o mesmo que será executado em tempo de execução, então você pode usar suas ferramentas e bibliotecas favoritas para desenvolver Workflows Temporal.

Temporal Workflows são resilientes. Eles podem executar e continuar executando por anos, mesmo que a infraestrutura subjacente falhe. Se a própria aplicação travar, o Temporal recriará automaticamente seu estado anterior à falha para que ela possa continuar exatamente de onde parou.

O que é um Workflow?

Uma sequência de etapas

Conceitualmente, um workflow define uma sequência de etapas. Com o Temporal, essas etapas são definidas escrevendo código, conhecido como Workflow Definition, e são executadas ao rodar esse código, o que resulta em uma Workflow Execution.

Exemplos de Workflows

Analisar alguns casos de uso potenciais nos ajuda a mapear conceitos para aplicações do mundo real.

Você provavelmente encontra workflows todos os dias. Alguns exemplos incluem assinar um serviço de entretenimento, comprar ingressos para um show, reservar uma viagem, pedir uma pizza ou registrar um relatório de despesas.

Você consegue pensar em outros workflows que encontrou recentemente? Como desenvolvedor, quais workflows você já ajudou a criar?

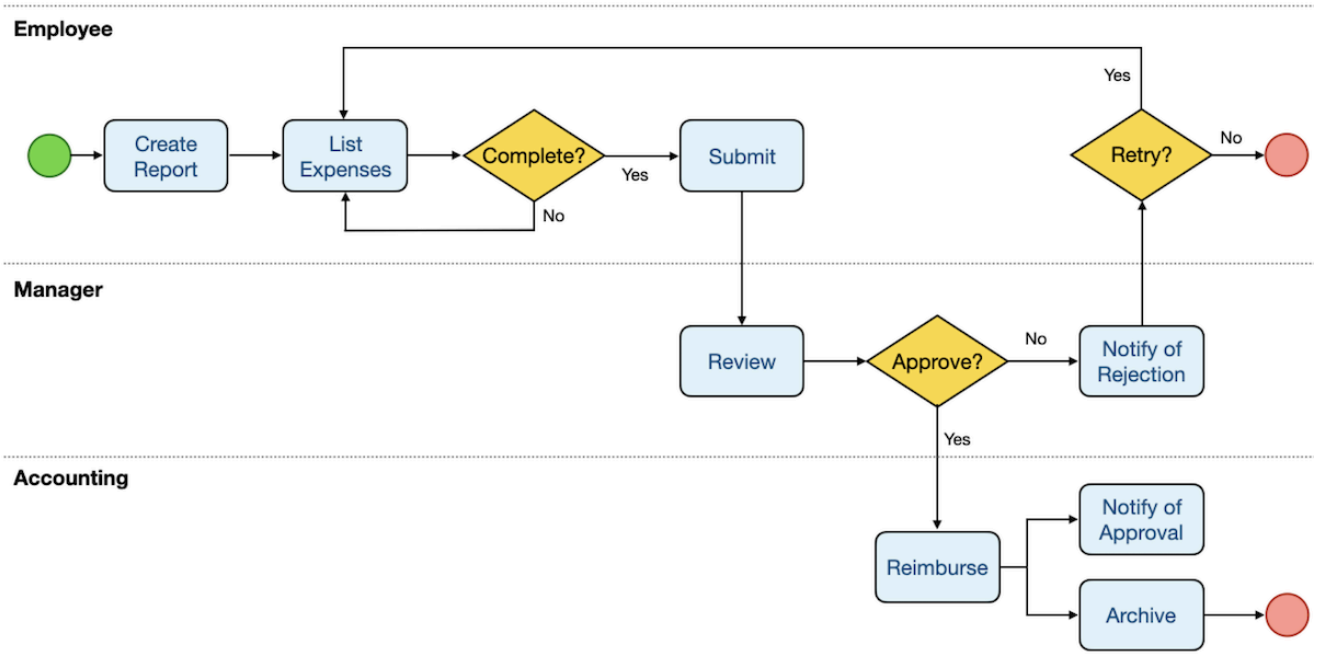
Exemplos de Workflows

Relatório de Despesas

Vamos analisar mais de perto o último exemplo.

Assim como os outros workflows, registrar um relatório de despesas envolve uma sequência de etapas. Primeiro, você cria o relatório, descreve os itens adquiridos, anexa recibos se necessário e então o submete. O gestor então o revisa, podendo rejeitá-lo (caso em que você será notificado para corrigir o problema e reenviar, se necessário) ou aprová-lo. Se

aprovado, o departamento de contabilidade o processará, realizará o reembolso por cheque ou depósito direto e então informará que o procedimento foi concluído. O relatório também será arquivado para estar disponível em caso de auditoria.



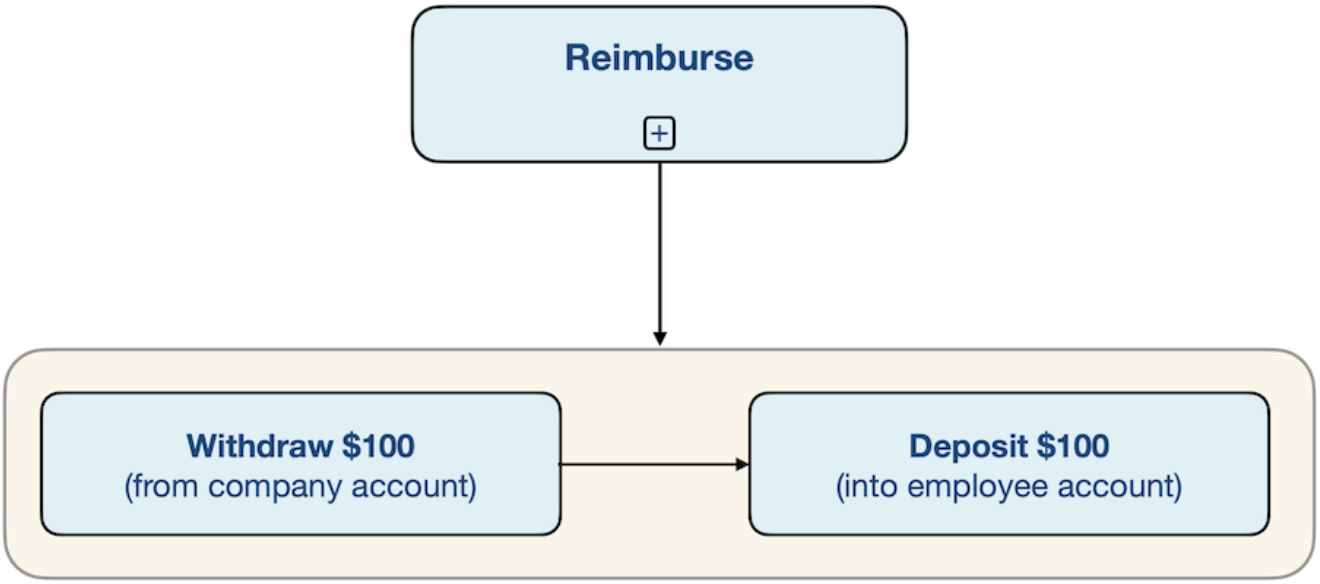
Esse workflow possui características interessantes, muitas das quais também estão presentes em workflows de outros domínios. Uma delas é que se trata de um processo potencialmente de longa duração. Dependendo da organização e do número de aprovações necessárias, pode levar dias, semanas ou mais do início ao fim. Outra característica é a presença de lógica condicional: assim como um programa de computador, há pontos de decisão e caminhos de execução que divergem conforme o resultado. Se o relatório for aceito, o reembolso é a próxima etapa; se for rejeitado, a próxima etapa é o envio de uma notificação solicitando que você o corrija e reenvie. Isso introduz outra característica: o workflow pode conter ciclos, já que um relatório rejeitado pode levar à correção, reenvio e nova revisão. Vale também destacar que o workflow envolve múltiplos pontos de interação humana, do funcionário, do gestor e do departamento de contabilidade, além de sistemas externos, notadamente o banco da empresa (origem do reembolso) e o banco do funcionário (destino dos fundos).

Transferência de Dinheiro

Outra característica interessante de um workflow é que ele pode ser composto por outros workflows. Vamos explorar um desses exemplos.

No cenário do relatório de despesas, você pode pensar no reembolso como uma única etapa, mas na prática ele é composto por duas operações distintas: a primeira é o saque da conta bancária do empregador e a segunda é o depósito do mesmo valor na conta bancária do funcionário. Há duas restrições importantes para que isso seja feito corretamente: as duas operações devem ser executadas e cada uma delas deve ser executada exatamente

uma vez.



De forma mais ampla, o reembolso é apenas uma transferência de dinheiro entre duas contas, e há inúmeros outros casos de uso para esse mesmo workflow. Milhões de pessoas dependem dele todos os dias ao utilizar serviços como Square, Stripe, Western Union, PayPal, Venmo, Swish ou Apple Pay.

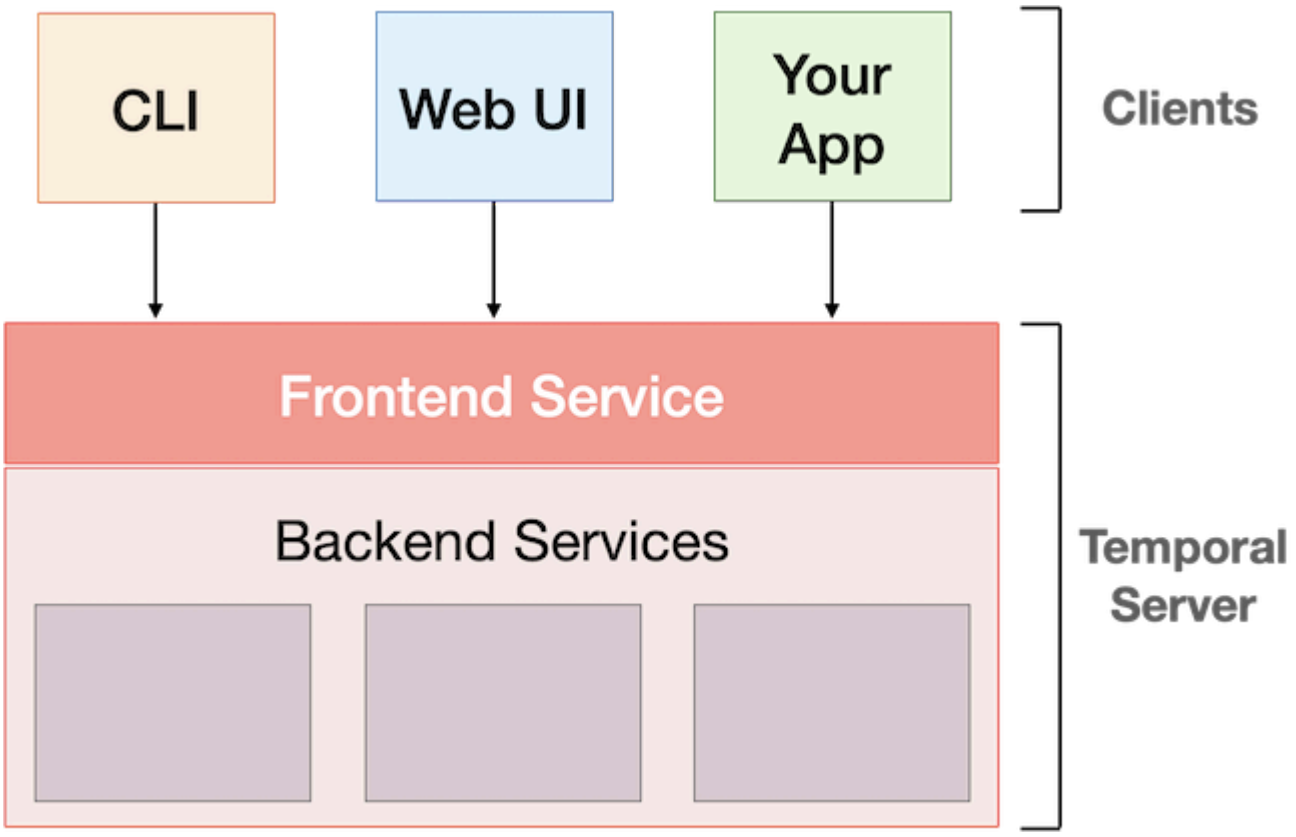
Esse workflow normalmente envolve múltiplas contas acessadas por meio de chamadas de procedimento remoto, caracterizando-o como um sistema distribuído. Como em qualquer sistema distribuído, ele pode falhar por diversas razões, incluindo falhas de servidor ou quedas de rede. Se esse workflow não fosse construído sobre o Temporal, as consequências poderiam ser catastróficas. Uma falha entre as etapas de saque e depósito deixaria os saldos incorretos e, como o estado atual seria perdido, reiniciar a aplicação repetiria o saque em vez de retomar pelo depósito. Como desenvolvedor, é sua responsabilidade detectar e mitigar essas falhas, mas ao usar Temporal Workflows, você pode deixar que a plataforma cuide desses tipos de problemas.

Visão Geral da Arquitetura

Temporal Server

Embora este treinamento seja voltado para desenvolvedores, é útil ter uma compreensão básica da arquitetura do Temporal. O nome "Temporal" pode ser ambíguo porque é tanto o nome do software quanto o nome da empresa fundada por seus criadores. No entanto, quando alguém menciona o Temporal sem contexto adicional, provavelmente está se referindo à Temporal Platform.

Você pode pensar na Temporal Platform como tendo duas partes, assim como a World Wide Web tem duas partes.



De um lado, há o servidor. O Temporal Server consiste em um serviço de frontend mais diversos serviços de backend que trabalham juntos para gerenciar a execução do código da sua aplicação. Todos esses serviços são escaláveis horizontalmente e um ambiente de produção normalmente executa múltiplas instâncias de cada um, distribuídas entre várias máquinas, para aumentar o desempenho e a disponibilidade.

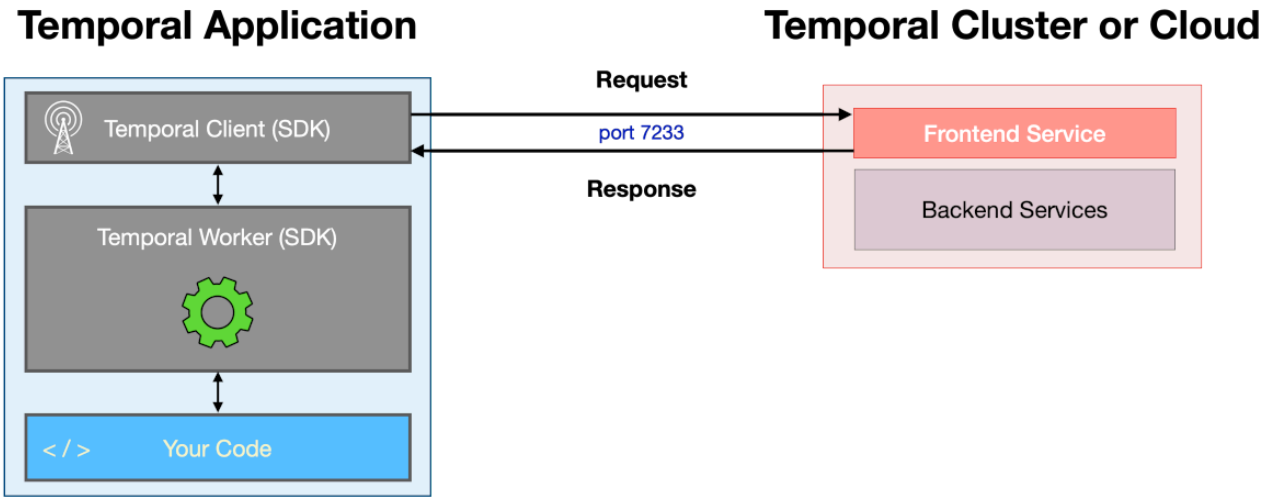
Do outro lado, há clientes que se comunicam com o Temporal Server. Neste curso, você trabalhará com três tipos de clientes:

- A interface de linha de comando do Temporal (CLI)
- A interface web do Temporal (Web UI)
- Um Temporal Client embutido nas aplicações que você executa

Vale destacar que o serviço de frontend do Temporal Server atua como um API gateway, ou seja, é um frontend para clientes, não para usuários finais (os usuários finais interagem via CLI ou Web UI).

Os clientes se comunicam com o Temporal Server enviando requisições ao Frontend Service, que então se comunica com os serviços de backend conforme necessário para atender à requisição e retorna uma resposta ao cliente. A comunicação com o Cluster e dentro dele é feita via gRPC, um framework RPC open source de alto desempenho originalmente desenvolvido pelo Google e atualmente parte do ecossistema da Cloud Native Computing Foundation. As mensagens são codificadas usando Protocol Buffers, um

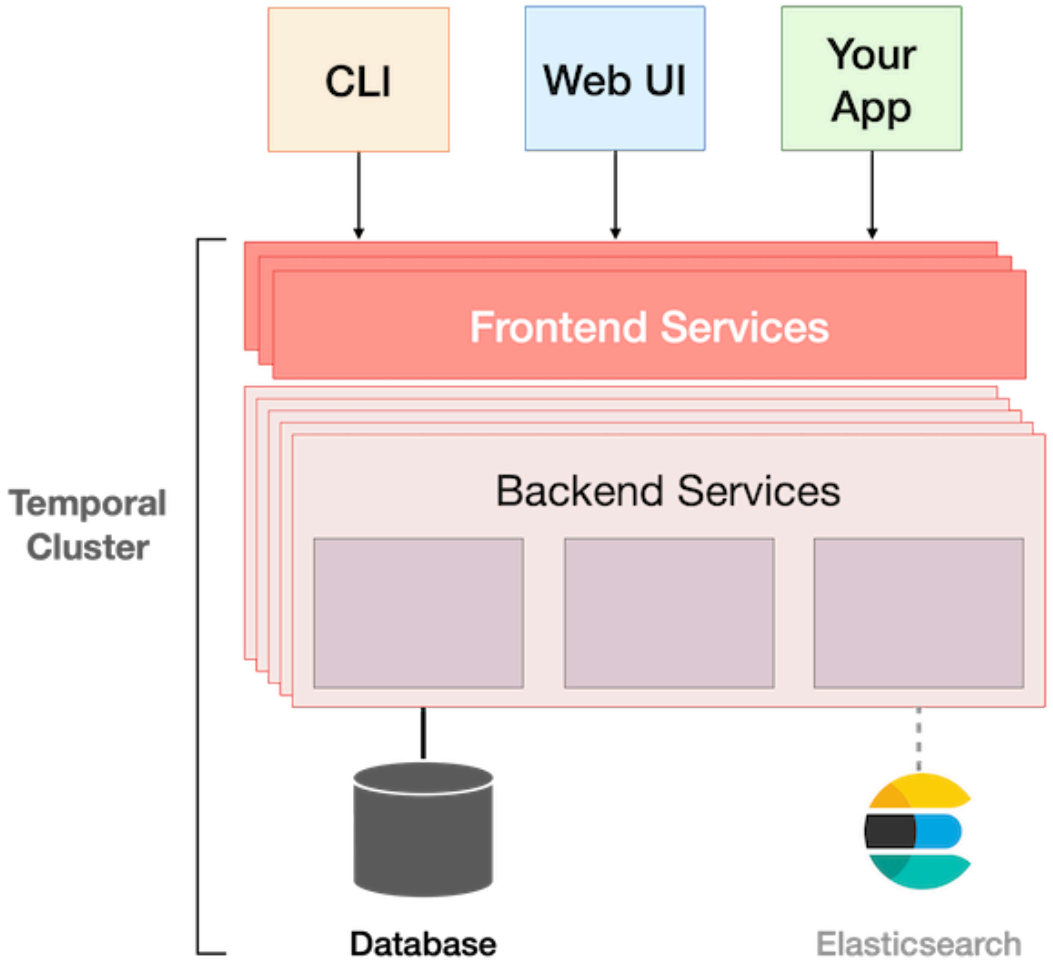
mecanismo de serialização open source também originalmente desenvolvido pelo Google.



Toda essa comunicação pode ser protegida com TLS, que criptografa os dados durante a transmissão pela rede e também pode verificar a identidade do cliente e do servidor por meio da validação de certificados.

Temporal Cluster

Assim como a CPU em um computador ou o motor em um carro, o Temporal Server é uma parte essencial do sistema, mas requer componentes adicionais para operar. O sistema completo é conhecido como Temporal Cluster, que é uma implantação do software Temporal Server em um conjunto de máquinas, junto com os componentes adicionais utilizados com ele.



O único componente obrigatório é um banco de dados, como Apache Cassandra, PostgreSQL ou MySQL. O Temporal Cluster rastreia o estado atual de cada execução dos seus Workflows e mantém um histórico de todos os Events que ocorrem durante as execuções, que utiliza para reconstruir o estado atual em caso de falha. Ele persiste essas e outras informações, como detalhes relacionados a timers duráveis e filas, no banco de dados.

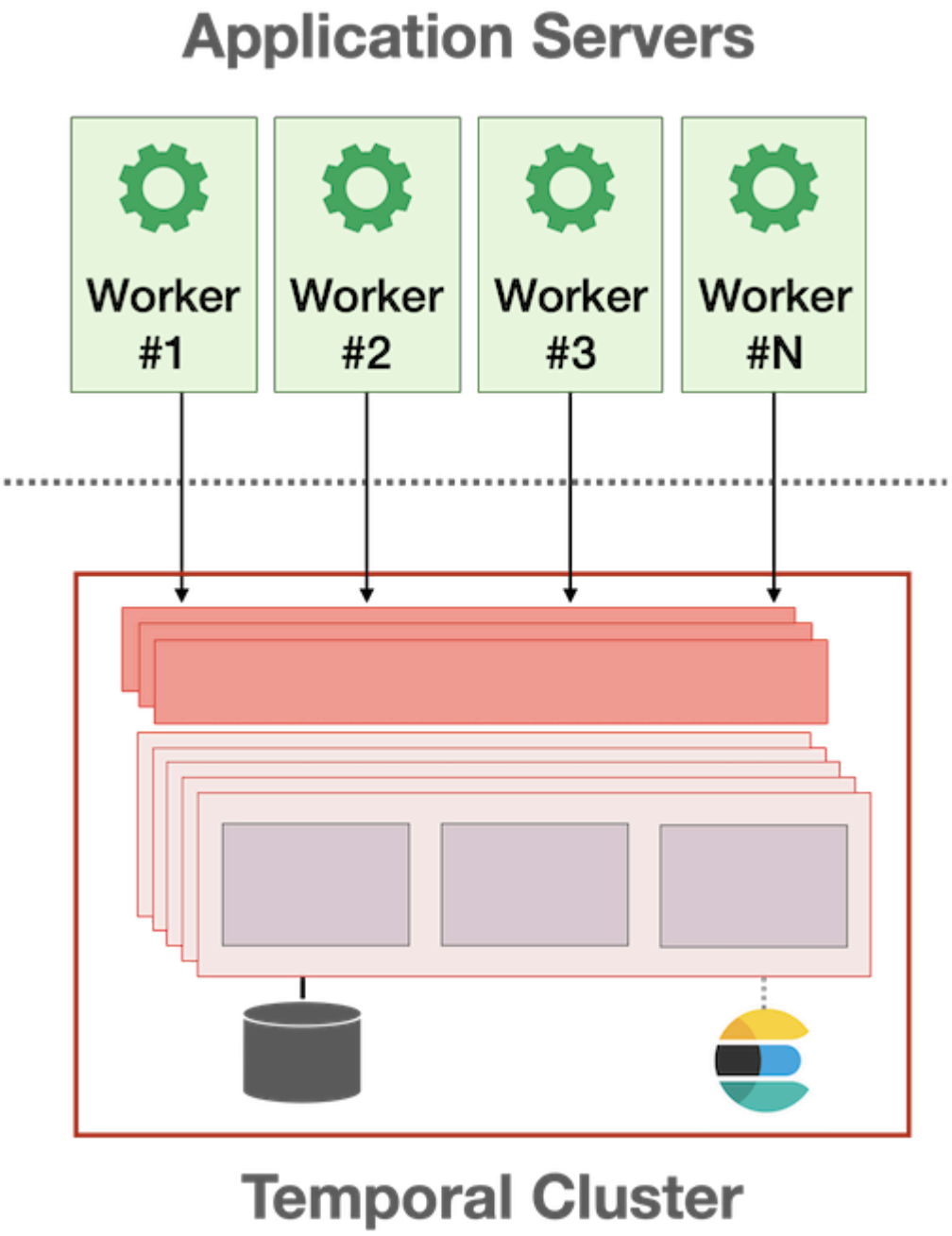
O Elasticsearch é um componente opcional. Não é necessário para a operação básica, mas adicioná-lo oferece capacidades avançadas de busca, ordenação e filtragem de informações sobre Workflow Executions atuais e recentes. Isso é útil quando você executa Workflows milhões de vezes e precisa localizar um específico, por exemplo, com base em quando foi iniciado, quanto tempo levou para executar ou seu status final.

Duas outras ferramentas são frequentemente utilizadas com o Temporal. O Prometheus é usado para coletar métricas do Temporal, enquanto o Grafana é usado para criar dashboards com base nessas métricas. Juntas, essas ferramentas ajudam as equipes de operações a monitorar a saúde do cluster e da aplicação.

Workers

Uma coisa que pode surpreender quem está chegando ao Temporal é que o Temporal Cluster não executa o seu código. Embora a plataforma garanta a execução durável do código, ela alcança isso por meio de orquestração. A execução do código da aplicação é externa ao cluster e, em implantações típicas, ocorre em um conjunto separado de servidores, potencialmente em um datacenter diferente do Temporal Cluster.

A entidade responsável por executar o seu código é conhecida como Worker, e é comum executar Workers em múltiplos servidores, pois isso aumenta tanto a escalabilidade quanto a disponibilidade da aplicação. O Worker, que faz parte da sua aplicação, se comunica com o Temporal Cluster para gerenciar a execução dos seus Workflows.



A aplicação conterá o código usado para inicializar o Worker, o Workflow e outras funções que compõem a lógica de negócio, e possivelmente também código usado para iniciar ou verificar o status do Workflow. Em tempo de execução, tudo que for necessário para executar a aplicação, incluindo bibliotecas e outras dependências referenciadas no código, precisará estar disponível em cada máquina onde pelo menos um processo Worker será

executado. O Temporal usa gRPC para comunicação, portanto cada máquina que executar um Worker precisará de conectividade com o Frontend Service do cluster Temporal.

Conectividade do Worker

Como o Worker usa um Temporal Client para se comunicar com o Temporal Cluster, cada máquina que executar um Worker precisará de conectividade com o Frontend Service do Cluster, que escuta na porta TCP 7233 por padrão.

Opções para Executar um Temporal Cluster

Existem várias formas de executar um Temporal Cluster, mas elas se enquadram em duas categorias: hospedar você mesmo ou deixar que o Temporal faça isso por você.

Self-Hosted

Uma opção para implantar um Temporal Cluster self-hosted é usar o Docker Compose. É extremamente conveniente para clusters de desenvolvimento porque elimina a necessidade de instalar e configurar manualmente os componentes individuais. O Temporal mantém um repositório no GitHub com diversas configurações prontas para uso.

Outra opção, descrita na seção "Configurando um Ambiente de Desenvolvimento Local" do capítulo anterior, é o suporte nativo do comando `temporal` para executar um servidor de desenvolvimento. Ele roda em um único processo e não possui dependências externas em tempo de execução, sendo menos complexo e menos exigente em recursos do que o Docker Compose.

Clusters Temporal self-hosted frequentemente são executados no Kubernetes, embora isso não seja obrigatório. A documentação oficial fornece mais informações sobre implantação de clusters.

Temporal Cloud

A alternativa a hospedar seu próprio Temporal Cluster é utilizar o Temporal Cloud, um serviço de nuvem totalmente gerenciado operado pelo time do Temporal. É uma forma simples, segura e escalável de sustentar suas aplicações Temporal, oferecendo 99,9% de uptime e conformidade com SOC2, além de suporte para desenvolvedores e ambientes de produção pelos especialistas do Temporal.

Usar o Temporal Cloud libera sua organização da carga operacional de manter seu próprio cluster, o que envolve não apenas o planejamento e a implantação inicial, mas também o trabalho contínuo de monitoramento, atualização e escalabilidade.

O Temporal Cloud utiliza precificação baseada em consumo, então você paga apenas pelo que usa, e pode visualizar seu uso atual e histórico a qualquer momento diretamente pela

interface web.

Onde Seu Código Executa

Para encerrar a discussão sobre as opções de implantação, vale reforçar um ponto importante abordado anteriormente. Independentemente de você hospedar seu próprio Temporal Cluster ou usar o Temporal Cloud, sua aplicação roda em servidores que você controla. Esses podem ser servidores no seu próprio datacenter ou máquinas virtuais hospedadas pelo seu provedor de nuvem preferido, mas é fundamental entender que o Temporal não executa o seu código e nem mesmo tem acesso a ele.

Integrando o Temporal em Outras Aplicações

Durante este curso, você interagirá principalmente com Workflow Executions por meio de uma interface de linha de comando ou de código executado no terminal. Embora essa abordagem permita iterar rapidamente durante o aprendizado, ela não necessariamente reflete a maioria das interações com Temporal Workflows no mundo real.

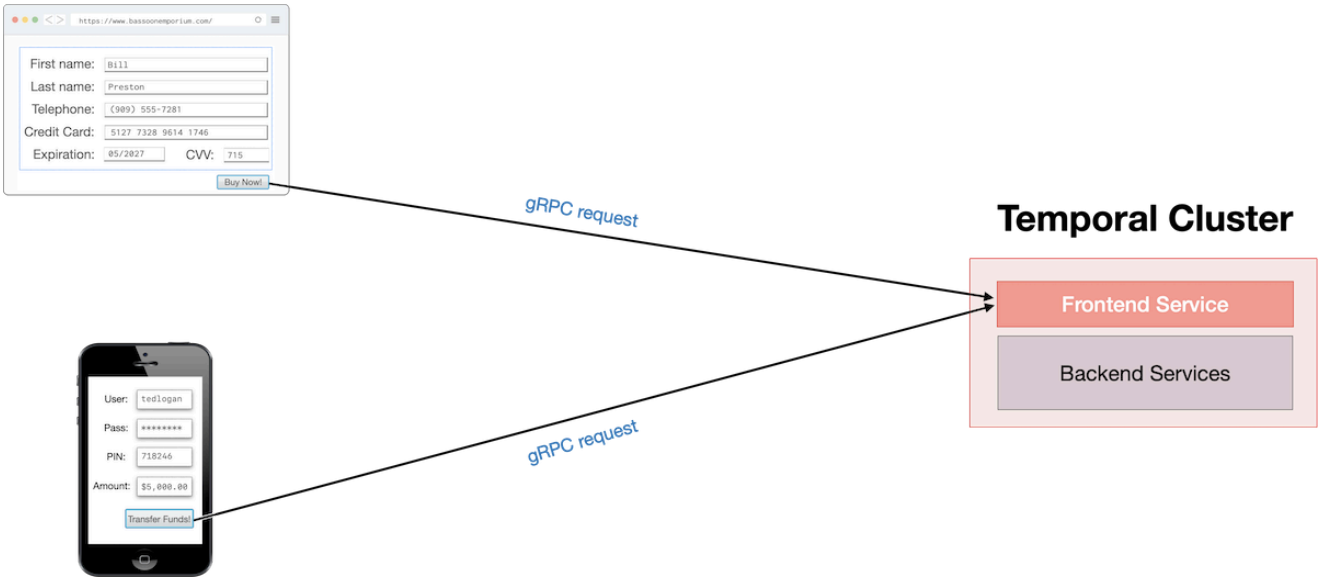
O Temporal atende a uma ampla variedade de casos de uso, como garantir que pedidos de e-commerce e transações financeiras sejam executados de forma confiável. Os usuários finais dessas aplicações não são desenvolvedores e provavelmente desconhecem o Temporal, mas suas ações disparam Workflow Executions e outras interações com o Temporal Cluster.

Isso levanta uma questão: como integrar a aplicação Temporal dentro da aplicação como um todo? Por exemplo, como iniciar uma Workflow Execution em resposta ao clique de um botão em um app web ou mobile?

Integração Direta no Frontend da Aplicação

É possível usar um Temporal Client diretamente nessas aplicações, assim como é possível fazer requisições gRPC diretamente sem utilizar um Temporal Client. No entanto, ambas as

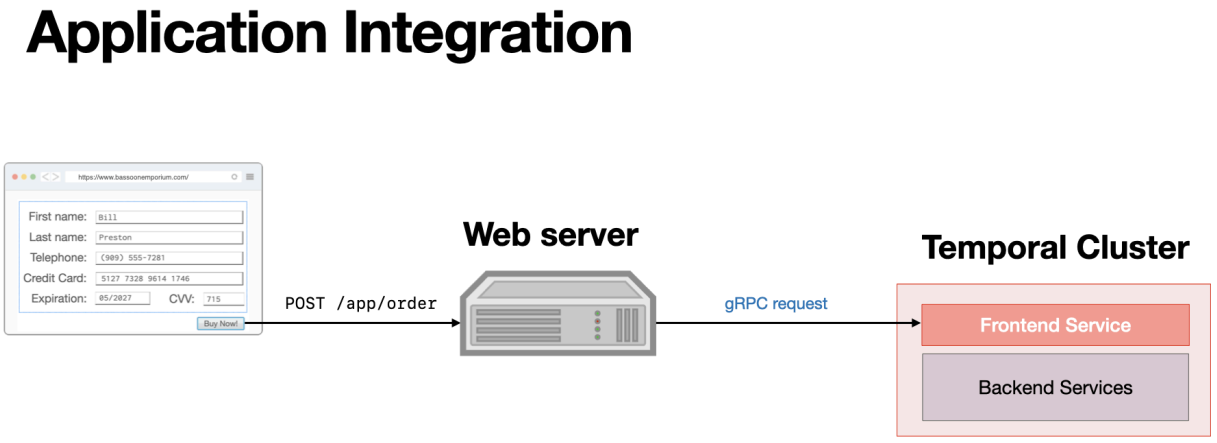
abordagens são atípicas.



Integração por meio de uma Aplicação Backend

Uma abordagem mais comum é fazer com que o app web ou mobile realize chamadas a um serviço, como uma aplicação web que expõe um endpoint REST, que atua como um gateway da aplicação.

Por exemplo, quando uma requisição é feita ao endpoint associado ao processamento de pedidos, o código dentro desse serviço pode extrair os dados da requisição HTTP e usá-los como entrada para o Workflow ao chamar o método de execução do Temporal Client. Isso, por sua vez, emite uma requisição gRPC ao Temporal Cluster, que inicia a Workflow Execution. De forma similar, pode haver endpoints para cancelar o Workflow ou recuperar seu resultado.



Essa abordagem é mais fácil de suportar do ponto de vista de segurança de rede, já que o Frontend Service do Temporal Cluster precisa aceitar conexões de entrada apenas do

servidor web, em vez de aceitar conexões de todos os usuários finais.

Temporal SDKs

O que é um SDK?

Você desenvolve Temporal Workflows escrevendo código em uma linguagem de programação padrão, de forma similar a como escreveria qualquer outro tipo de aplicação. Esse código fará chamadas às APIs do Temporal, que por sua vez usam um Temporal Client para se comunicar com o cluster. A biblioteca que fornece suporte para isso é chamada de Software Development Kit, ou SDK.

O Temporal oferece SDKs para diversas linguagens de programação, incluindo Go, Java, TypeScript, PHP, .NET e Python. SDKs para outras linguagens também estão planejados para o futuro.

Como Instalar o Python SDK

O ambiente de exercícios fornecido para este curso já inclui o Python SDK, portanto você não precisará instalá-lo durante o curso.

No entanto, você precisará dele para desenvolver suas próprias aplicações Temporal após o término do curso. Você pode configurá-lo na sua máquina executando o comando abaixo a partir do diretório raiz do seu módulo, ou seja, aquele que contém o arquivo de dependências do projeto, como o diretório `edu-101-python-code` utilizado neste curso.

Este comando é fornecido para referência futura e não é necessário executá-lo agora. Antes de instalar qualquer pacote Python, certifique-se de criar um ambiente virtual.

```
(env) $ pip install temporalio
```

Vale observar que os passos para instalar SDKs para outras linguagens variarão, geralmente seguindo as convenções de gerenciamento de pacotes de cada linguagem. Consulte a documentação para detalhes sobre como instalar SDKs para outras linguagens.

Interface de Linha de Comando do Temporal: `temporal`

O que é o `temporal`?

O Temporal fornece uma interface de linha de comando (CLI), o `temporal`, que permite interagir com um cluster, iniciar um servidor de desenvolvimento e muito mais. Durante este

curso, você usará o `temporal` para iniciar Workflows e visualizar o status e o histórico de suas execuções, embora ele possua muitas outras capacidades avançadas.

Executar o comando sem nenhum argumento exibirá informações sobre o uso básico:

```
$ temporal
The Temporal CLI manages, monitors, and debugs Temporal apps. It lets you
run
a local Temporal Service, start Workflow Executions, pass messages to
running
Workflows, inspect state, and more.

* Start a local development service:
      temporal server start-dev
* View help: pass --help to any command:
      temporal activity complete --help

Usage:
  temporal [command]

Available Commands:
  activity    Complete or fail an Activity
  batch       Manage running batch jobs
  completion  Generate the autocompletion script for the specified shell
  env         Manage environments
  help        Help about any command
  operator    Manage Temporal deployments
  schedule    Perform operations on Schedules
  server      Run Temporal Server
  task-queue  Manage Task Queues
  workflow    Start, list, and operate on Workflows
```

O programa possui 10 comandos, cada um com vários subcomandos. Felizmente, você não precisa memorizar todos eles. Adicionando `--help` após um comando ou parte de um comando, você verá uma ajuda mais específica:

```
$ temporal workflow --help
Workflow commands perform operations on Workflow Executions:

temporal workflow [command] [options]

For example:

temporal workflow list

Usage:
  temporal workflow [command]
```

```
Available Commands:
  cancel           Send cancellation to Workflow Execution
  count            Number of Workflow Executions
  delete           Remove Workflow Execution
  describe         Show Workflow Execution info
  execute          Start new Workflow Execution
...
Use "temporal workflow [command] --help" for more information about a
command.
```

Como Instalar o `temporal`

O ambiente de exercícios fornecido para este curso já possui o `temporal` instalado. No entanto, você pode querer instalá-lo na sua própria máquina para usá-lo com seus próprios clusters no futuro.

Os passos de instalação variam de acordo com o gerenciador de pacotes disponível na sua máquina, a disponibilidade do Docker e sua preferência por compilar a partir do código-fonte.

Se você estiver usando uma máquina com o gerenciador de pacotes Homebrew, comum em Macs, pode usar o seguinte comando:

```
$ brew install temporal
```

A documentação do Temporal descreve métodos alternativos para instalar o Temporal CLI, incluindo instruções para Linux e Windows.

Definindo um Workflow em Python

No Temporal, você define um Workflow em Python criando uma classe. O código que compõe essa classe é conhecido como Workflow Definition.

Começando pela lógica de negócio

O primeiro passo é escrever a lógica de negócio, sem nenhum código específico do Temporal. O método abaixo recebe uma string com o nome de uma pessoa como parâmetro de entrada e retorna outra string com uma saudação personalizada.

O Python SDK do Temporal utiliza a funcionalidade nativa `asyncio` da biblioteca padrão do Python, por isso definimos o método como `async`. O uso de métodos não-async é suportado, mas requer uma configuração mais complexa com múltiplos processos. Por essa razão, recomenda-se o uso de `async`.

```
class GreetSomeone:
    async def run(self, name: str) -> str:
        return f"Hello {name}!"
```

Essa é a lógica de negócio do nosso primeiro Workflow, mas ainda não é um Workflow: por ora, é apenas uma classe Python.

Testando a lógica de negócio

Podemos testar o método escrevendo um pequeno programa Python para invocá-lo. Esse padrão corresponde ao que você fará com o Temporal, onde o Workflow contém a lógica de negócio, tipicamente separada do código usado para executá-la.

O programa abaixo lê a entrada de um argumento da linha de comando, invoca o método da lógica de negócio passando esse valor, atribui a string retornada a uma variável chamada `greeting` e a imprime na saída padrão.

```
import sys
import asyncio
from workflow import GreetSomeone


async def main():
    name = sys.argv[1]
    greeter = GreetSomeone()
    greeting = await greeter.run(name)
    print(greeting)

if __name__ == "__main__":
    asyncio.run(main())
```

Transformando a classe em um Workflow Definition

O próximo passo é transformar esse método em um Temporal Workflow Definition. O Temporal não impõe regras sobre como nomear o método do Workflow, portanto não há necessidade de renomear a classe já escrita.

Todo Workflow possui um nome, que o Temporal chama de Workflow Type. Pode ser um termo confuso, mas pense nele como um "tipo" em uma linguagem de programação, uma forma de se referir a uma entidade no código, como um tipo "Customer" ou "Product". No Python SDK, por padrão, o Workflow Type é o nome da classe usada para definir o Workflow, mas é possível sobrescrevê-lo e fornecer um valor mais amigável, já que a Web UI exibe Workflow Executions pelo seu tipo.

Transformar o método em um Workflow Definition requer apenas três etapas:

1. Importar o módulo `workflow` do Python SDK do Temporal
2. Decorar a classe com o decorator `@workflow.defn`
3. Decorar o método com o decorator `@workflow.run`

```
from temporalio import workflow


@workflow.defn
class GreetSomeone:
    @workflow.run
    async def run(self, name: str) -> str:
        return f"Hello {name}!"
```

O método acima é agora um Workflow Definition e é o mesmo código que você usará para executar o Workflow no primeiro exercício prático.

Parâmetros de Entrada e Valores de Retorno

Os valores precisam ser serializáveis

O Temporal mantém informações sobre Workflow Executions atuais e passadas. Um dos benefícios disso é que você pode usar a Web UI para explorar esses detalhes ao investigar um problema, mesmo que tenha ocorrido vários dias antes. No entanto, isso também afeta como você projeta suas Workflow Definitions.

Para que o Temporal armazene a entrada e a saída do Workflow, os dados usados em parâmetros de entrada e valores de retorno precisam ser serializáveis. Por padrão, o Temporal suporta valores nulos ou binários, além de qualquer dado que possa ser serializado em JSON. Isso significa que a maioria dos tipos comumente usados em funções, como inteiros, números de ponto flutuante, booleanos e strings, são tratados automaticamente, assim como `dataclasses` compostas por esses tipos. Já tipos como `datetime`, funções ou outros tipos não serializáveis são proibidos como parâmetros de entrada ou valores de retorno.

Confidencialidade dos dados

Embora os parâmetros de entrada e valores de retorno sejam armazenados como parte do Event History das suas Workflow Executions, você pode criar um Data Converter personalizado para criptografar os dados ao entrarem no Temporal Cluster e descriptografá-los na saída, mantendo a confidencialidade de informações sensíveis. Data Converters personalizados estão além do escopo do curso Temporal 101, mas a documentação fornece mais informações e você pode consultar exemplos no GitHub.

Evite passar grandes quantidades de dados

Como o Event History contém a entrada e a saída, que também são transmitidas pela rede da aplicação para o Temporal Cluster, você terá melhor desempenho limitando a quantidade de dados enviados. Por exemplo, imagine que você criou um Workflow para converter arquivos de áudio de um formato para outro. Seria muito mais eficiente passar o caminho ou a URL dos arquivos como entrada do que passar o conteúdo dos arquivos em si.

Para proteger contra falhas inesperadas causadas pelo envio ou armazenamento de dados em excesso, o Temporal Server impõe diversos limites a partir dos quais emitirá avisos ou erros, dependendo da gravidade. A documentação possui uma página com detalhes sobre esses limites.

Inicializando o Worker

O papel de um Worker

Como mencionado anteriormente, os Workers executam o código dos seus Workflows. O Worker em si é fornecido pelo Temporal SDK, mas sua aplicação incluirá código para configurá-lo e executá-lo. Quando esse código é executado, o Worker estabelece uma conexão persistente com o Temporal Cluster e começa a fazer polling em uma Task Queue no Cluster, buscando trabalho a realizar.

Como os Workers executam o seu código, qualquer Workflow que você iniciar não avançará a menos que pelo menos um Worker esteja em execução.

Inicializando um Worker

Geralmente, há três elementos necessários para configurar um Worker:

1. Um Temporal Client, usado para se comunicar com o Temporal Cluster
2. O nome de uma Task Queue, mantida pelo Temporal Server e monitorada pelo Worker via polling
3. A classe do Workflow Definition, usada para registrar a implementação do Workflow no Worker

Código de inicialização do Worker

A seguir, um exemplo de programa Python usado para inicializar e iniciar um Worker capaz de executar o Workflow Definition definido em `greeting.py`:

```
import asyncio

from temporalio.client import Client
from temporalio.worker import Worker

from greeting import GreetSomeone
```

```
async def main():
    client = await Client.connect("localhost:7233", namespace="default")
    # Run the worker
    worker = Worker(client, task_queue="greeting-tasks", workflows=
[GreetSomeone])
    await worker.run()


if __name__ == "__main__":
    asyncio.run(main())
```

Explicação da inicialização

A função `main` começa criando um cliente, `client`, fornecendo o endereço e a porta do cluster Temporal, além do nome do namespace. O Worker usará esse cliente para se comunicar com o cluster.

Em seguida, o código usa o `client` criado para instanciar um `Worker`, passando também o nome da Task Queue e o Workflow Definition. A classe do Workflow Definition é importada do arquivo que a implementa e passada como argumento nomeado ao Worker. Isso informa ao Worker qual Task Queue monitorar e qual Workflow está registrado para ser executado por ele.

Após a configuração, o método `run()` do Worker é aguardado com `await` para iniciá-lo. O Worker então começa um "long poll" na Task Queue especificada. Se você iniciar o Worker a partir de um terminal, não se surpreenda se não houver nenhuma saída: esse é o comportamento esperado. O programa não está travado, está apenas fazendo polling na Task Queue e processando as tarefas recebidas do Temporal Cluster.

O ciclo de vida de um Worker

O tempo de vida do Worker e a duração de uma Workflow Execution são independentes. A função `worker.run()` é bloqueante e só para se for encerrada ou encontrar um erro fatal. O processo do Worker pode durar dias, semanas ou mais. Se os Workflows que ele processa forem relativamente curtos, um único Worker pode executar milhares ou mesmo milhões deles ao longo de sua vida útil.

Por outro lado, um Workflow pode rodar por anos, enquanto o servidor onde o processo do Worker está em execução pode ser reiniciado após alguns meses por um administrador realizando manutenção. Se o Workflow Type estiver registrado em outros Workers, um ou mais deles continuarão automaticamente de onde o Worker original parou. Se não houver outros Workers disponíveis, a Workflow Execution continuará de onde parou assim que o Worker original for reiniciado. Em ambos os casos, o tempo de inatividade não causará falha na Workflow Execution.

Escolhendo nomes para Task Queues

Nomes de Task Queue são sensíveis a maiúsculas e minúsculas. Você reduzirá a probabilidade de problemas escolhendo nomes descritivos, curtos e simples. Por exemplo, `greeting-tasks` é mais fácil de entender e lembrar do que `gtq`, e também é uma escolha melhor do que `task-queue-name-for-the-greeting-workflow`, por ser mais curto e menos suscetível a erros de digitação.

Executando um Workflow pela Linha de Comando

Usando o `temporal` para iniciar um Workflow

Você já aprendeu como criar um Workflow Definition e inicializar um Worker capaz de executá-lo. O próximo passo é rodar sua aplicação.

Uma forma de iniciar o Workflow é usando a ferramenta de linha de comando `temporal` com um comando similar ao abaixo:

```
$ temporal workflow start \
    --type GreetSomeone \
    --task-queue greeting-tasks \
    --workflow-id my-first-workflow \
    --input '"Mason"'
```

Note o uso de aspas no valor de entrada: aspas duplas dentro de aspas simples. A entrada passada ao comando `temporal` deve estar em formato JSON, e essa combinação de aspas é necessária para que o valor seja transmitido corretamente pelo shell ao Workflow.

Executando no Windows

A combinação de aspas simples e duplas descrita acima é específica para shells no estilo UNIX. Se você estiver usando o Temporal CLI no Windows (como no PowerShell), utilize o escape de aspas no estilo Windows:

```
temporal workflow start --type GreetSomeone --task-queue greeting-tasks --
workflow-id my-first-workflow --input '\"Mason"'
```

Essa é uma abordagem geral do Windows para tratar aspas em parâmetros, não algo específico do Temporal.

Explicação dos argumentos do comando

O primeiro argumento é o Workflow Type. No Python SDK, esse valor corresponde por padrão ao nome da classe decorada com `@workflow.defn`.

O segundo é o nome da Task Queue que o Temporal Cluster usará, que deve corresponder exatamente ao valor fornecido na inicialização do Worker. Como Task Queues são criadas dinamicamente, digitar o nome incorretamente não causará um erro, mas resultará em duas Task Queues diferentes. Como o Cluster e o Worker não compartilhariam a mesma fila nesse caso, a Workflow Execution nunca avançaria.

O comando também especifica um Workflow ID, que é opcional, mas recomendado. Trata-se de um identificador definido pelo usuário, geralmente com algum significado de negócio. Por exemplo, um Workflow de relatório de despesas poderia ter um Workflow ID que identifica o relatório ou o funcionário que o submeteu. Se omitido, um UUID será atribuído automaticamente.

Como esse Workflow requer entrada, o comando fornece o valor correspondente. Ao submeter um Workflow pela linha de comando, a entrada está sempre em formato JSON, por isso as aspas duplas aparecem dentro das aspas simples. Digitar JSON diretamente na linha de comando funciona bem para casos simples como esse, mas seria impraticável para dados mais complexos. Nesses casos, você pode salvar a entrada em um arquivo JSON e especificar seu caminho com a opção `--input-file`, em vez de usar `--input` para fornecer os dados inline.

O que acontece ao executar o comando

Ao executar o comando, ele submete a requisição de execução ao cluster, que responde com o Workflow ID fornecido, ou um UUID atribuído automaticamente caso ele tenha sido omitido.

O comando também exibe um Run ID, que identifica de forma única essa execução específica do Workflow. O resultado retornado pelo Workflow não é exibido, pois Workflows podem durar meses ou anos. O resultado desse comando é:

```
Running execution:
  WorkflowId  my-first-workflow
  RunId       019db723-f40e-7e51-b06f-29f942d61728
  Type        GreetSomeone
  Namespace   default
  TaskQueue   greeting-tasks
```

Para recuperar o resultado, use o comando `temporal workflow show`:

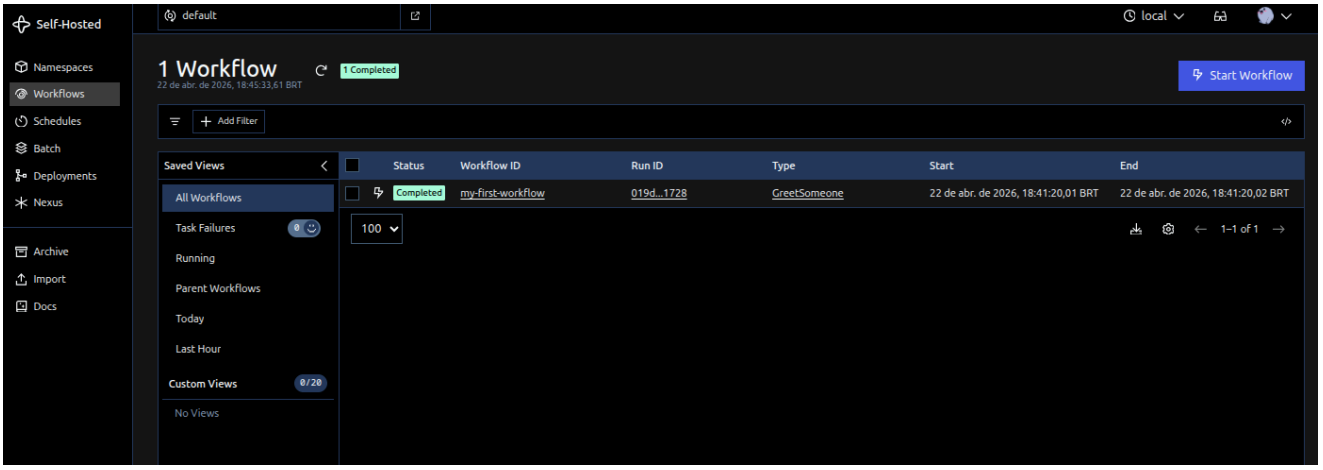
```
$ temporal workflow show --workflow-id my-first-workflow
Progress:
  ID           Time                     Type
    1  2026-04-22T21:41:20Z  WorkflowExecutionStarted
    2  2026-04-22T21:41:20Z  WorkflowTaskScheduled
    3  2026-04-22T21:41:20Z  WorkflowTaskStarted
    4  2026-04-22T21:41:20Z  WorkflowTaskCompleted
```

```
    5  2026-04-22T21:41:20Z  WorkflowExecutionCompleted

Results:
  Status          COMPLETED
  Result          "Hello Mason!"
  ResultEncoding  json/plain
```

Interface de Usuário

Acessando http://localhost:8080/namespaces/default/workflows, temos:



Exercício Prático #1: Hello Workflow

Durante este exercício, você irá:

- Revisar a lógica de negócio do Workflow Definition fornecido para entender seu comportamento
- Modificar o código de inicialização do Worker para especificar o nome de uma Task Queue (`greeting-tasks`)
- Executar o código de inicialização do Worker para iniciar o processo do Worker
- Usar o `temporal` para executar o Workflow pela linha de comando, passando seu nome como entrada

Se você ainda não iniciou o ambiente de exercícios, faça isso agora.

Consulte o arquivo `README.md` no ambiente de exercícios para detalhes sobre este exercício. Você encontrará o código no diretório `exercises/hello-workflow`. Lembre-se de fazer suas alterações no subdiretório `practice`.

Os comentários `TODO` indicarão onde as alterações são necessárias. Se precisar de uma dica ou quiser comparar suas mudanças, consulte a versão completa no subdiretório `solution`.

Executando um Workflow a partir do Código da Aplicação

Usando código para iniciar um Workflow

No exercício anterior, você usou o `temporal` para iniciar um Workflow pela linha de comando. Uma alternativa é iniciar o Workflow a partir do código da sua aplicação, que é a abordagem usada nos próximos exercícios práticos para evitar a necessidade de digitar comandos longos repetidamente.

Embora ambas as abordagens produzam o mesmo resultado, fazê-lo via código oferece uma forma de integrar o Temporal às suas próprias aplicações. Por exemplo, você pode executar ou encerrar um Workflow em resposta a alguma ação do usuário, como o clique de um botão em um app web ou mobile.

Exemplo de código

```
import asyncio
import sys

from greeting import GreetSomeone
from temporalio.client import Client


async def main():
    # Create client connected to server at the given address
    client = await Client.connect("localhost:7233")

    # Execute a workflow
    handle = await client.start_workflow(
        GreetSomeone.run,
        sys.argv[1],
        id="greeting-workflow",
        task_queue="greeting-tasks",
    )

    print(f"Started workflow. Workflow ID: {handle.id}, RunID
{handle.result_run_id}")

    result = await handle.result()

    print(f"Result: {result}")


if __name__ == "__main__":
    asyncio.run(main())
```

Explicação: execução do Workflow

A aplicação segue três etapas principais para iniciar o Workflow:

1. Importar a classe `Client` do Temporal SDK
2. Criar e configurar um cliente
3. Usar a API para solicitar a execução

O código usado para criar e configurar o cliente aqui é idêntico ao usado durante a inicialização do Worker. Você pode estruturar sua aplicação de forma que o mesmo cliente seja compartilhado entre essas duas partes do código. Neste curso, no entanto, o código de inicialização do Worker e o código de início do Workflow foram mantidos separados para facilitar a distinção do papel de cada um.

As opções de Workflow Execution especificam o Workflow ID e o nome da Task Queue, os mesmos dois valores fornecidos como argumentos ao comando `temporal`.

A aplicação solicita a execução do Workflow chamando o método `start_workflow` do cliente, passando o método do Workflow, a entrada, o Workflow ID e o nome da Task Queue. Ao iniciar um Workflow a partir do código, não é necessário fornecer a entrada em formato JSON como na linha de comando. Você pode usar qualquer tipo permitido, como inteiros, strings ou dataclasses, e o SDK fará a conversão para JSON automaticamente.

Explicação: recuperando o resultado

Lembre-se que Workflows podem rodar por longos períodos. A chamada a `start_workflow` não aguarda a conclusão do Workflow, portanto a linha que exibe o Workflow ID e o Run ID será executada uma fração de segundo depois, mesmo que o Workflow leve anos para concluir.

Não é obrigatório aguardar ou recuperar o resultado, mas o exemplo acima demonstra como fazê-lo. Como o resultado só estará disponível após a conclusão da Workflow Execution, `start_workflow` retorna um `WorkflowHandle` que fornece acesso ao resultado assim que ele estiver disponível.

A chamada `handle.result()` bloqueará até que a Workflow Execution seja concluída. Se ela for concluída com sucesso, a variável `result` receberá o valor de saída. Se for concluída com falha, uma exceção será lançada.

Caso você queira bloquear o programa no momento da invocação do Workflow, pode condensar o código e usar o método `execute_workflow`, tornando o Workflow síncrono:

```
import asyncio
import sys

from greeting import GreetSomeone
from temporalio.client import Client
```

```python
async def main():
    # Create client connected to server at the given address
    client = await Client.connect("localhost:7233")

    # Execute a workflow
    result = await client.execute_workflow(
        GreetSomeone.run,
        sys.argv[1],
        id="greeting-workflow",
        task_queue="greeting-tasks",
    )

    print(f"Result: {result}")


if __name__ == "__main__":
    asyncio.run(main())
```

Neste curso, usaremos Workflow Executions assíncronas e o método `start_workflow`.

Visualizando o Histórico de Workflows com o `temporal` CLI

Executando `temporal workflow show`

O Temporal Cluster mantém informações detalhadas sobre o histórico de cada Workflow Execution. Essa é mais uma vantagem de desenvolver na Temporal Platform, pois oferece visibilidade sobre o que está acontecendo nas suas aplicações, tanto as que estão em execução quanto as que foram executadas recentemente.

Uma forma de visualizar o histórico de uma Workflow Execution é executando um comando similar ao abaixo:

```
$ temporal workflow show --workflow-id my-first-workflow
```

Interpretando a saída do comando

Executar o comando acima produzirá uma saída similar a esta, que detalha os eventos ocorridos durante a execução do Workflow:

```
Progress:
  ID           Time                     Type
    1  2024-08-20T19:22:22Z  WorkflowExecutionStarted
    2  2024-08-20T19:22:22Z  WorkflowTaskScheduled
    3  2024-08-20T19:22:22Z  WorkflowTaskStarted
    4  2024-08-20T19:22:22Z  WorkflowTaskCompleted
```

```
    5  2024-08-20T19:22:22Z  WorkflowExecutionCompleted
Results:
  Status          COMPLETED
  Result          "Hello Mason!"
  ResultEncoding  json/plain
```

Essa Workflow Execution possui cinco itens em seu Event History, além do status e da saída da execução. Você aprenderá mais sobre esses Events adiante no curso.

Para ver um histórico mais detalhado, execute o seguinte comando:

```
$ temporal workflow show --workflow-id my-first-workflow --detailed
```

```
Progress:

identity: temporal-cli:masonegger@Masons-Laptop
input[0]: Mason
originalExecutionRunId: 86afe3ea-8563-4056-9de1-e8ed6ca5b1e3
taskId: 1048587
taskQueue.kind: TASK_QUEUE_KIND_NORMAL
taskQueue.name: greeting-tasks
workflowExecutionTimeout: 0s
workflowId: my-first-workflow
workflowRunTimeout: 0s
workflowTaskTimeout: 10s
workflowType.name: GreetSomeone

--------------- [2] WorkflowTaskScheduled ---------------
attempt: 1
eventTime: 2024-08-20T19:22:22.077680Z
startToCloseTimeout: 10s
taskId: 1048588
taskQueue.kind: TASK_QUEUE_KIND_NORMAL
taskQueue.name: greeting-tasks

--------------- [3] WorkflowTaskStarted ---------------
eventTime: 2024-08-20T19:22:22.081601Z
historySizeBytes: 304
identity: 35461@Masons-Laptop
requestId: a0e88f09-ebc0-4b4d-bf1a-4c8fc616859c
scheduledEventId: 2
taskId: 1048593
workerVersion.buildId: a29a35df004d1a3519db4896d1335c96

--------------- [4] WorkflowTaskCompleted ---------------
eventTime: 2024-08-20T19:22:22.089363Z
identity: 35461@Masons-Laptop
scheduledEventId: 2
```

```
  sdkMetadata.coreUsedFlags[0]: 1
  sdkMetadata.coreUsedFlags[1]: 2
  startedEventId: 3
  taskId: 1048597
  workerVersion.buildId: a29a35df004d1a3519db4896d1335c96

  --------------- [5] WorkflowExecutionCompleted ---------------
  eventTime: 2024-08-20T19:22:22.089482Z
  result[0]: Hello Mason!
  taskId: 1048598
  workflowTaskCompletedEventId: 4

  Results:
    Status          COMPLETED
    Result          "Hello Mason!"
    ResultEncoding  json/plain
```

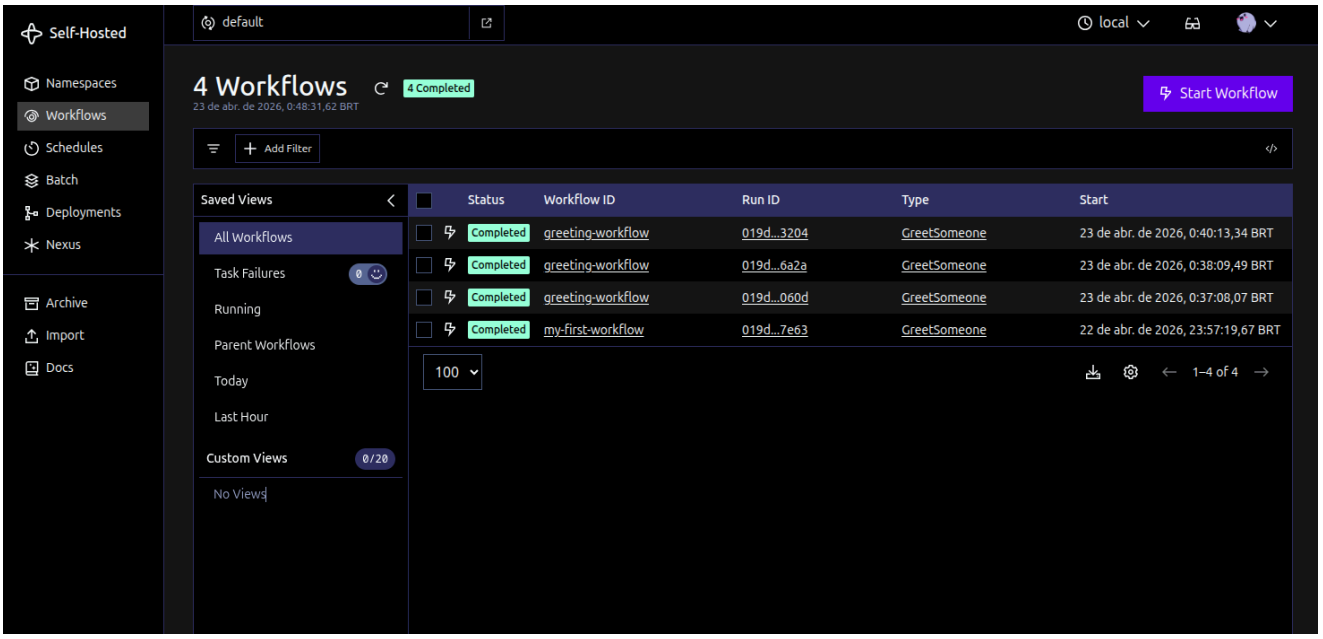
Os mesmos eventos da saída anterior aparecem aqui com mais contexto. Os detalhes exibidos ao lado do primeiro Event identificam o Workflow Type (`GreetSomeone`), a Task Queue (`greeting-tasks`), o valor de entrada (`Mason`) e as configurações de timeout usadas para essa Workflow Execution. Os três eventos seguintes indicam que o Temporal Cluster agendou um Workflow Task, que foi então iniciado e concluído por um Worker. O evento final confirma que a Workflow Execution foi concluída, retornando o resultado (`Hello Mason!`).

Visualizando o Histórico de Workflows pela Web UI

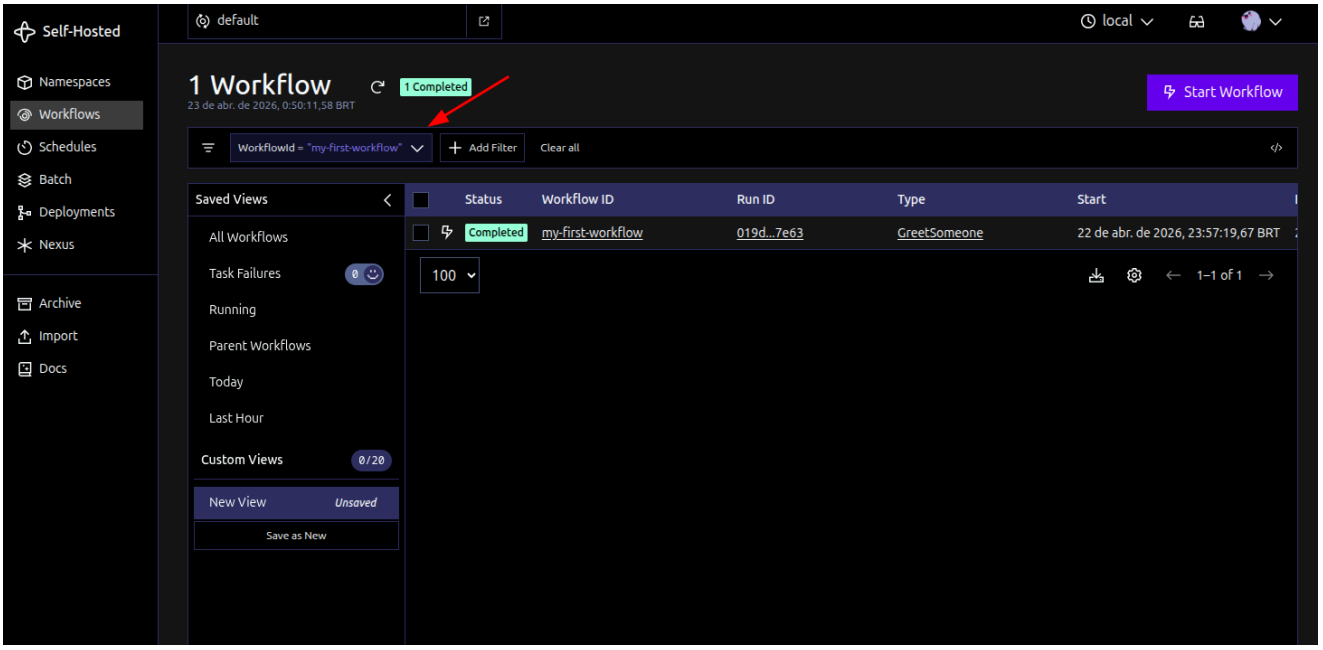
O método de acesso à Web UI varia conforme o tipo de implantação. Se você estiver usando o Temporal Cloud, o acesso é feito por meio de uma conexão segura na porta HTTPS padrão em `cloud.temporal.io`, exigindo login. Se você usou o `temporal` para implantar um cluster de desenvolvimento localmente, pode acessá-la pelo `localhost` na porta 8233, embora as instruções anteriores tenham configurado a flag para disponibilizá-la na porta 8080. Em um cluster self-hosted de produção, consulte o administrador para obter o hostname e a porta.

Página principal

A página principal da Web UI exibe Workflow Executions recentes. À esquerda, há uma barra de ferramentas para navegar para outras páginas, embora no curso Temporal 101 você utilize principalmente esta página.

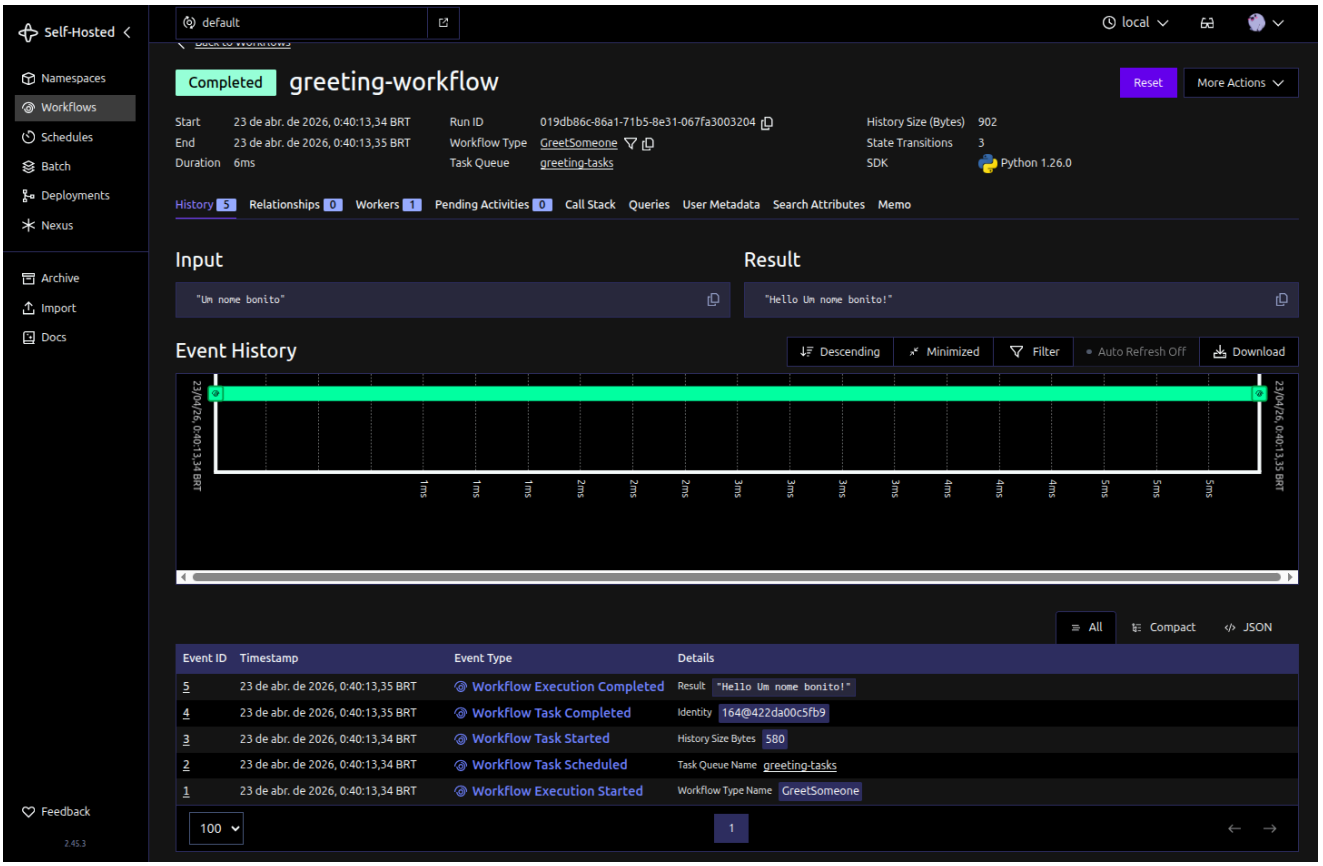


A maior parte da página é ocupada por uma tabela que lista Workflow Executions. Acima dessa tabela, há campos para filtrar por Workflow ID, Workflow Type, janela de tempo ou status de execução. À direita, é possível alterar o formato de exibição dos timestamps, alternando entre o fuso horário UTC padrão, o fuso horário local ou até um tempo relativo, como "32 minutos atrás".



Página de detalhes

Ao clicar em um item da tabela, a Web UI exibirá a página de detalhes da Workflow Execution selecionada. O Workflow ID é exibido no topo, seguido por outros detalhes como o Workflow Type e a Task Queue. Abaixo, à esquerda, são exibidos os dados de entrada do Workflow e, à direita, os dados de saída. Na parte inferior da página, o Event History mostra exatamente o que ocorreu em cada etapa da Workflow Execution.



Namespaces

A Web UI exibe Workflow Executions dentro de um namespace específico. Um namespace fornece um meio de isolamento dentro do Temporal, de forma similar a como namespaces ou pacotes em algumas linguagens de programação isolam diferentes partes do código. Eles permitem separar logicamente as coisas de acordo com qualquer critério que atenda às suas necessidades. Por exemplo, você pode ter um namespace "development" e um namespace "production", ou separar por equipe ou departamento.

Um aspecto importante desse isolamento é que algumas opções de configuração são aplicadas por namespace. Por exemplo, o Temporal garante que existe apenas uma Workflow Execution com um determinado Workflow ID em execução dentro de qualquer namespace. Se você tentar usar o mesmo Workflow ID para iniciar um Workflow enquanto outro já estiver em execução naquele namespace, dependendo da configuração, o sistema retornará informações sobre a execução em andamento ou um erro informando o conflito.

O namespace atual é exibido próximo ao topo da página principal. Clicar em "namespaces" na navegação à esquerda mostra os namespaces disponíveis e permite alternar entre eles. O namespace padrão é chamado `default`. Você pode usar o `temporal` para registrar namespaces adicionais, e o código especificará o namespace desejado por meio de uma opção de configuração fornecida ao criar um cliente.

Fazendo Alterações em um Workflow

A compatibilidade retroativa é uma consideração importante no Temporal. Você pode executar um determinado Workflow Definition centenas, milhares ou milhões de vezes. Se a execução falhar, o Temporal reconstruirá o estado do Workflow antes da falha e continuará a execução a partir desse ponto. Isso tem implicações sobre como você desenvolve e mantém os Workflow Definitions.

Parâmetros de entrada e valores de retorno

Em geral, você deve evitar alterar o número ou os tipos de parâmetros de entrada e valores de retorno do seu Workflow.

O Temporal recomenda que a função do Workflow receba um único parâmetro de entrada, um dataclass, em vez de múltiplos parâmetros. Adicionar campos opcionais ao dataclass não altera o tipo do dataclass em si, o que oferece uma forma retrocompatível de evoluir o código. Os exemplos usados neste curso não seguem essa recomendação por uma escolha deliberada, visando reduzir a complexidade e facilitar o acompanhamento por iniciantes em Python. No entanto, você verá esse padrão nos tutoriais oficiais do Temporal em Python.

Determinismo

Embora as aplicações Temporal sejam frequentemente usadas para gerenciar trabalhos não determinísticos, como chamadas a microsserviços ou LLMs, o Workflow Definition é o código determinístico usado para orquestrar esse trabalho. De forma simplificada, o determinismo é o requisito de que cada execução de um dado Workflow produza a mesma saída para a mesma entrada. Isso significa que você não deve, por exemplo, trabalhar com números aleatórios no código do Workflow.

E as operações não determinísticas?

O requisito de determinismo se aplica apenas à lógica de orquestração do Workflow. Praticamente tudo que interage com o mundo externo é inerentemente não determinístico, como chamadas a APIs de LLMs, consultas a bancos de dados, leitura ou escrita de arquivos e requisições HTTP a serviços externos.

A boa notícia é que sua aplicação Temporal pode lidar com todas essas operações. Se você precisar fazer algo não determinístico, basta mover esse código para uma função separada chamada **Activity**, referenciada no código do Workflow. As Activities são executadas fora do caminho de replay do Workflow e são reexecutadas de forma confiável pelo Temporal. Você aprenderá sobre Activities e começará a usá-las no próximo capítulo.

Isso significa que o Temporal não pode ser usado para IA?

Não, pelo contrário. O determinismo do Workflow é exatamente o que torna o Temporal uma ótima escolha para aplicações de IA. Como chamadas a LLMs, uso de ferramentas e etapas de agentes ficam em Activities, o Workflow pode orquestrar a sequência enquanto o Temporal persiste cada etapa. Isso permite que seu agente se recupere de falhas, tente novamente chamadas falhas a LLMs e retome tarefas de longa duração sem perder estado.

Essa separação entre Workflows determinísticos e Activities não determinísticas é o que permite ao Temporal fornecer Durable Execution. No curso Temporal 102, você aprenderá exatamente como isso funciona e verá como sua aplicação se recupera automaticamente de falhas.

Versionamento

Como Workflow Executions no Temporal podem rodar por longos períodos, às vezes meses ou até anos, é comum precisar fazer alterações significativas em um Workflow Definition mesmo enquanto uma execução está em andamento. O Versionamento é um recurso do Temporal que ajuda a gerenciar essas mudanças de forma segura. Com ele, você pode modificar o Workflow Definition para que novas execuções usem o código atualizado, enquanto as existentes continuam rodando a versão original. Isso é especialmente útil ao fazer uma alteração não determinística, ou seja, que muda a ordem de execução dentro de uma Workflow Execution em andamento. O SDK do Temporal permite rastrear e gerenciar essas versões, deixando execuções antigas usarem o código original enquanto novas execuções usam a versão modificada.

Saiba mais sobre Versionamento no nosso curso gratuito Versioning Workflows.

Reiniciando o Processo do Worker

Após fazer alterações na sua aplicação, você precisará implantá-las no servidor.

Com a maioria dos SDKs do Temporal, é necessário reiniciar o Worker para que as alterações no código entrem em vigor. Embora o Python SDK utilize um sandbox que recarrega automaticamente o Workflow do disco a cada execução, tornando esse reinício desnecessário, esse é um detalhe de implementação específico do Temporal Python SDK e pode mudar no futuro. Portanto, para garantir a execução correta, recomendamos parar o Worker e reiniciá-lo sempre que fizer alterações no código.

O que são Activities?

Você aprendeu anteriormente que o código de um Workflow deve ser determinístico e produzir a mesma saída para a mesma entrada. Isso também implica que ele não pode interagir com o mundo externo, como acessar arquivos ou recursos de rede, invocar LLMs ou outros serviços de IA, pois esses recursos podem não estar disponíveis em determinado momento ou retornar resultados diferentes a cada chamada. No entanto, sua lógica de negócio pode exigir exatamente esse tipo de operação. Como resolver isso?

No Temporal, você pode usar Activities para encapsular lógica de negócio sujeita a falhas. Ao contrário do Workflow Definition, não há requisito de determinismo para uma Activity

Definition.

Em geral, qualquer operação que introduza a possibilidade de falha deve ser executada como parte de uma Activity, e não diretamente no Workflow. Enquanto as Activities são executadas como parte de uma Workflow Execution, elas têm uma característica importante: são reexecutadas automaticamente em caso de falha. Se você tiver um Workflow extenso que precisa acessar um serviço e esse serviço ficar indisponível, não é desejável reexecutar o Workflow inteiro. Em vez disso, basta retentar apenas a parte que falhou, definindo esse código em uma Activity e referenciando-a no Workflow Definition. O código dentro dessa Activity Definition será executado, reexecutado se necessário, e o Workflow continuará seu progresso assim que a Activity for concluída com sucesso.

Visão geral da implementação de Activities

Uma Activity Definition em Python pode ser implementada de diferentes formas. Uma delas é implementar a Activity como uma função decorada com o decorator `@activity.defn`:

```
from temporalio import activity

@activity.defn
async def greet_in_french(name: str) -> str:
    return f"Bonjour {name}!"
```

As Activities em Python também podem ser implementadas como métodos dentro de uma classe, dependendo das suas necessidades. Por exemplo, quando é necessário passar uma sessão de cliente para realizar requisições HTTP:

```
class TranslateActivities:
    def __init__(self, session: aiohttp.ClientSession):
        self.session = session

    @activity.defn
    async def greet_in_spanish(self, name: str, stem: str) -> str:
        base = f"http://localhost:9999/{stem}"
        url = f"{base}?name={urllib.parse.quote(name)}"

        async with self.session.get(url) as response:
            translation = await response.text()

        return translation
```

As Activities seguem as mesmas regras de tipos permitidos como parâmetros de entrada e valores de retorno que o Workflow Definition. Por exemplo, qualquer tipo que converta para JSON é permitido, mas tipos como `datetime`, funções ou outros tipos não serializáveis não são.

O Temporal não impõe regras sobre como nomear essas funções. A recomendação é manter os Workflow Definitions em um arquivo separado do restante do código. Devido à implementação do Temporal Python SDK, o arquivo do Workflow Definition é recarregado a cada execução, portanto minimizar seu conteúdo reduzirá os recarregamentos e melhorará o desempenho.

Activities Assíncronas vs. Síncronas

O Temporal Python SDK suporta múltiplas formas de implementar uma Activity:

- Assincronamente com `asyncio`
- Sincronamente com múltiplas threads usando `concurrent.futures.ThreadPoolExecutor`
- Sincronamente com múltiplos processos usando `concurrent.futures.ProcessPoolExecutor` e `multiprocessing.managers.SyncManager`

É importante implementar suas Activities usando o método correto, caso contrário sua aplicação pode falhar de formas esporádicas e inesperadas. A escolha depende do seu caso de uso.

O event loop assíncrono do Python e chamadas bloqueantes

O event loop assíncrono do Python roda em uma thread e executa todas as tarefas nessa mesma thread. Quando qualquer tarefa está em execução no event loop, o loop fica bloqueado e nenhuma outra tarefa pode executar simultaneamente. Sempre que uma tarefa executa uma expressão `await`, ela é suspensa e o event loop inicia ou retoma a execução de outra tarefa.

Isso significa que o event loop só pode transferir o controle quando a palavra-chave `await` é executada. Se um programa faz uma chamada bloqueante, como leitura de arquivo, requisição síncrona a um serviço de rede ou qualquer outra operação que bloqueie a execução, o event loop inteiro precisa aguardar até que essa execução seja concluída.

Bloquear o event loop assíncrono transformaria seu programa assíncrono em um programa síncrono e serial, anulando completamente o propósito de usar `asyncio`. Isso também pode levar a deadlocks e comportamentos imprevisíveis. A depuração desses problemas pode ser difícil e demorada, pois localizar a origem da chamada bloqueante nem sempre é imediato.

Por isso, desenvolvedores Python devem ter cuidado extra para não fazer chamadas bloqueantes dentro de uma Activity assíncrona, ou usar uma biblioteca async-safe para realizar essas operações. Por exemplo, fazer uma requisição HTTP com a popular biblioteca `requests` dentro de uma Activity assíncrona bloquearia o event loop. Nesse

caso, use uma biblioteca async-safe como `aiohttp` ou `httpx`. Caso contrário, use uma Activity síncrona.

Implementando Activities assíncronas

O código abaixo é uma Activity Definition assíncrona que recebe um nome (`str`) como entrada e retorna uma saudação personalizada (`str`) como saída, fazendo uma chamada a um microsserviço via HTTP para obter a saudação em espanhol. Ela usa a biblioteca `aiohttp` para realizar a requisição de forma async-safe.

A Activity é implementada como uma classe porque a biblioteca `aiohttp` requer uma `Session` estabelecida para realizar requisições HTTP. Criar uma nova `Session` a cada invocação seria ineficiente, então o código aceita um objeto `Session` como parâmetro de instância e o disponibiliza aos métodos. Essa abordagem também facilita o encerramento da `Session` ao final da execução.

```
import aiohttp
import urllib.parse
from temporalio import activity


class TranslateActivities:
    def __init__(self, session: aiohttp.ClientSession):
        self.session = session

    @activity.defn
    async def greet_in_spanish(self, name: str) -> str:
        greeting = await self.call_service("get-spanish-greeting", name)
        return greeting

    async def call_service(self, stem: str, name: str) -> str:
        base = f"http://localhost:9999/{stem}"
        url = f"{base}?name={urllib.parse.quote(name)}"

        async with self.session.get(url) as response:
            translation = await response.text()

            if response.status >= 400:
                raise ApplicationError(
                    f"HTTP Error {response.status}: {translation}",
                    non_retryable=response.status < 500,
                )

            return translation
```

Implementando Activities síncronas

O código abaixo é uma implementação síncrona da mesma Activity. Ao fazer a chamada ao microsserviço, usa a biblioteca `requests`, o que é seguro em Activities síncronas.

```
import urllib.parse
import requests
from temporalio import activity


class TranslateActivities:

    @activity.defn
    def greet_in_spanish(self, name: str) -> str:
        greeting = self.call_service("get-spanish-greeting", name)
        return greeting

    def call_service(self, stem: str, name: str) -> str:
        base = f"http://localhost:9999/{stem}"
        url = f"{base}?name={urllib.parse.quote(name)}"

        response = requests.get(url)
        return response.text
```

Como neste exemplo não há compartilhamento de estado entre invocações, o `__init__` foi removido. Sem a necessidade de uma classe, as Activities podem ser implementadas simplesmente como funções decoradas:

```
@activity.defn
def greet_in_spanish(name: str) -> str:
    greeting = call_service("get-spanish-greeting", name)
    return greeting

def call_service(stem: str, name: str) -> str:
    base = f"http://localhost:9999/{stem}"
    url = f"{base}?name={urllib.parse.quote(name)}"

    response = requests.get(url)
    return response.text
```

A escolha entre implementar Activities como classe ou funções é uma decisão de design do desenvolvedor quando não há estado compartilhado entre Activities. Ambas as abordagens são igualmente válidas.

Quando usar Activities assíncronas

Activities assíncronas têm diversas vantagens, como potencial ganho de velocidade na execução. No entanto, fazer chamadas não seguras dentro do event loop assíncrono pode causar bugs esporádicos e difíceis de diagnosticar. Por esse motivo, recomendamos usar

Activities assíncronas apenas quando você tiver certeza de que suas Activities são async-safe e não fazem chamadas bloqueantes.

Se você encontrar bugs que suspeita serem resultado de uma chamada não segura em uma Activity assíncrona, converta-a para uma Activity síncrona e verifique se o problema é resolvido.

O restante deste curso demonstrará Activities assíncronas. Se quiser concluir o Exercício 3 usando Activities síncronas, você pode trocar de branch no repositório `edu-101-python-code` para o branch `sync`.

Registrando Activities

Assim como você precisa registrar seus Workflows ao inicializar o Worker, também é necessário realizar uma etapa similar para as Activities. O processo de registro é quase idêntico ao de um Workflow.

Se você estiver registrando uma Activity implementada como função, importe-a do arquivo de origem e passe-a como argumento nomeado, de forma similar ao registro de um Workflow:

```
from temporalio.client import Client
from temporalio.worker import Worker

from translate import greet_in_spanish
from greeting import GreetSomeone

...
    worker = Worker(
        client,
        task_queue="greeting-tasks",
        workflows=[GreetSomeone],
        activities=[greet_in_spanish],
    )
...
```

Registro de Activity assíncrona e exemplo de Worker

O código de inicialização do Worker abaixo demonstra o registro de uma Activity assíncrona implementada como classe, definida em um arquivo separado chamado `translate.py`. Neste exemplo, uma `aiohttp.ClientSession` chamada `session` é criada usando um context manager. Context Managers são uma ferramenta útil em Python para garantir que os recursos sejam liberados ao sair do escopo. Em seguida, cria-se uma instância da classe da Activity, passando a `session`, e a Activity é registrada usando o nome do método:

```python
import asyncio
import aiohttp

from temporalio.client import Client
from temporalio.worker import Worker

from translate import TranslateActivities
from greeting import GreetSomeone


async def main():
    client = await Client.connect("localhost:7233", namespace="default")

    async with aiohttp.ClientSession() as session:
        activities = TranslateActivities(session)

        worker = Worker(
            client,
            task_queue="greeting-tasks",
            workflows=[GreetSomeone],
            activities=[activities.greet_in_spanish],
        )
        await worker.run()


if __name__ == "__main__":
    asyncio.run(main())
```

Modificando o Worker para executar Activities síncronas

Ao executar Activities síncronas, é necessário passar um `activity_executor` ao Worker. O Temporal Python SDK suporta atualmente Thread Pools ou Multiprocessing como executores de Activity. A escolha depende do seu caso de uso e implantação.

Ao usar o `ThreadPoolExecutor`, primeiro importe `concurrent.futures` e crie um objeto `ThreadPoolExecutor` especificando o número máximo de workers. Esse número deve ser compatível com o `max_concurrent_activities` do Worker, que por padrão é 100. Portanto, defina `max_workers` com pelo menos 100. Um valor menor poderia fazer com que o Worker aceitasse tarefas mas fosse incapaz de processá-las, potencialmente causando timeouts.

Se usar um context manager para criar o `ThreadPoolExecutor`, atribua o objeto a uma variável com o `as`. Por fim, passe o `ThreadPoolExecutor` ao Worker pelo argumento nomeado `activity_executor`:

```python
import concurrent.futures
import asyncio
```

```
from temporalio.client import Client
from temporalio.worker import Worker

from translate import TranslateActivities
from greeting import GreetSomeone


async def main():
    client = await Client.connect("localhost:7233", namespace="default")

    activities = TranslateActivities()

    with concurrent.futures.ThreadPoolExecutor(max_workers=100) as
activity_executor:
        worker = Worker(
            client,
            task_queue="greeting-tasks",
            workflows=[GreetSomeone],
            activities=[activities.greet_in_spanish,
activities.farewell_in_spanish],
            activity_executor=activity_executor,
        )
        print("Starting the worker....")
        await worker.run()


if __name__ == "__main__":
    asyncio.run(main())
```

Executando Activities assíncronas e síncronas no mesmo Worker

Pode haver situações em que uma Activity precise fazer uma chamada bloqueante que não pode ser feita de forma async-safe. Isso não significa que todas as Activities precisam ser síncronas. Você pode implementar apenas essa Activity como síncrona e passá-la ao mesmo Worker que executa suas Activities assíncronas.

O exemplo abaixo implementa `greet_in_spanish` como Activity assíncrona e `thank_you_in_spanish` como Activity síncrona:

```
import urllib.parse
import requests
import aiohttp
from temporalio import activity


class TranslateActivities:
```

```
    @activity.defn
    async def greet_in_spanish(self, name: str) -> str:
        greeting = await self.call_service_async("get-spanish-greeting",
name)
        return greeting

    @activity.defn
    def thank_you_in_spanish(self, name: str) -> str:
        thank_you = self.call_service_sync("get-spanish-thank-you", name)
        return thank_you

    async def call_service_async(self, stem: str, name: str) -> str:
        # implementação omitida por brevidade
        ...

    def call_service_sync(self, stem: str, name: str) -> str:
        # implementação omitida por brevidade
        ...
```

O Worker correspondente seria implementado da seguinte forma:

```
import concurrent.futures
import aiohttp
import asyncio

from temporalio.client import Client
from temporalio.worker import Worker

from translate import TranslateActivities
from greeting import GreetSomeone


async def main():
    client = await Client.connect("localhost:7233", namespace="default")

    async with aiohttp.ClientSession() as session:
        activities = TranslateActivities(session)
        with concurrent.futures.ThreadPoolExecutor(max_workers=100) as
activity_executor:
            worker = Worker(
                client,
                task_queue="greeting-tasks",
                workflows=[GreetSomeone],
                activities=[activities.greet_in_spanish,
activities.thank_you_in_spanish],
                activity_executor=activity_executor,
            )
            print("Starting the worker....")
            await worker.run()
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Executando Activities

As Activities do Temporal são executadas a partir de dentro de um Workflow. O código a seguir seria escrito na implementação do Workflow Definition.

Importando módulos em arquivos de Workflow

No Temporal Python SDK, os arquivos de Workflow são recarregados em um sandbox a cada execução. Para evitar o recarregamento de um import a cada execução, você pode marcá-lo como "pass through", reutilizando o módulo de fora do sandbox. Módulos da biblioteca padrão e módulos do `temporalio` são passados por padrão. Todos os outros módulos usados de forma determinística, como referências a funções de Activity ou módulos de terceiros, devem ser passados dessa forma.

Por esse motivo, recomendamos manter os arquivos de Workflow separados do restante do código, incluindo tipos de entrada/saída e Activities.

Este trecho deve ser incluído no seu Workflow abaixo dos imports da biblioteca padrão e do `temporalio`:

```
# Import activity, passing it through the sandbox without reloading the
module
with workflow.unsafe.imports_passed_through():
    from translate import TranslateActivities
```

Especificando opções de Activity

Um passo essencial para executar uma Activity dentro do seu Workflow é especificar as opções que governam sua execução:

```
async def run(self, name: str) -> str:
    greeting = await workflow.execute_activity_method(
        TranslateActivities.greet_in_spanish,
        name,
        start_to_close_timeout=timedelta(seconds=5),
    )

    return greeting
```

No exemplo acima, a opção `start_to_close_timeout` foi definida como cinco segundos. Seu valor deve ser maior do que o tempo máximo que você estima para a execução da Activity. Isso permite que o Temporal Cluster detecte um Worker que travou, considerando aquela tentativa como falha e criando uma nova tarefa que outro Worker pode assumir.

É obrigatório definir o `start_to_close_timeout` ou o `schedule_to_close_timeout`. Neste curso, usaremos o `start_to_close_timeout`.

Executando a Activity

Use a função `workflow.execute_activity_method` para solicitar a execução da Activity. O primeiro argumento é a função ou método que define a Activity. Se a Activity recebe um parâmetro de entrada, ele é fornecido como segundo argumento. Os demais argumentos são as opções desejadas:

```
greeting = await workflow.execute_activity_method(
    TranslateActivities.greet_in_spanish,
    name,
    start_to_close_timeout=timedelta(seconds=5),
)
```

`workflow.execute_activity_method` deve ser usado quando a Activity é implementada como classe. Quando implementada como função, usa-se `execute_activity`.

Recuperando o resultado

O Workflow não executa a Activity diretamente, ou seja, não invoca a função da Activity. Em vez disso, faz uma requisição ao Temporal Cluster, pedindo que agende a execução da Activity.

A chamada a `execute_activity_method` é bloqueante e atribui o resultado da Activity à variável `greeting` assim que ela é concluída.

Embora `execute_activity_method` seja uma chamada síncrona, você também pode executar Activities de forma assíncrona usando `start_activity_method`, que retorna um `ActivityHandle`. A partir daí, você pode aguardar o handle quando estiver pronto para recuperar o resultado:

```
greeting_handle = workflow.start_activity_method(
    TranslateActivities.greet_in_spanish,
    name,
    start_to_close_timeout=timedelta(seconds=5)
)

greeting = await greeting_handle
```

Neste curso, usaremos `execute_activity_method`.

Juntando tudo

O exemplo abaixo mostra o Workflow Definition completo que solicita a execução de duas Activities e recupera seus resultados:

```
from datetime import timedelta
from temporalio import workflow

# Import activity, passing it through the sandbox without reloading the
module
with workflow.unsafe.imports_passed_through():
    from translate import TranslateActivities


@workflow.defn
class GreetSomeone:
    @workflow.run
    async def run(self, name: str) -> str:
        greeting = await workflow.execute_activity_method(
            TranslateActivities.greet_in_spanish,
            name,
            start_to_close_timeout=timedelta(seconds=5),
        )

        farewell = await workflow.execute_activity_method(
            TranslateActivities.farewell_in_spanish,
            name,
            start_to_close_timeout=timedelta(seconds=5),
        )

        return f"{greeting}\n{farewell}"
```

Usando Timeouts Apropriados

Como desenvolvedores, frequentemente aprendemos começando por um exemplo funcional e realizando uma série de iterações, fazendo modificações e observando seus efeitos. Mesmo após dominar a tecnologia, às vezes é mais fácil iniciar um novo projeto copiando código existente para ganhar tempo, em vez de implementar tudo do zero.

Isso pode economizar tempo no desenvolvimento, mas causar problemas em produção quando seus requisitos diferem do original. Esse é um problema particularmente comum para desenvolvedores Temporal quando copiam código de Workflow ou Activity de um exemplo mas esquecem de ajustar os valores de Timeout para seu caso de uso real.

Por exemplo, o `start_to_close_timeout` representa o tempo permitido para a execução de uma Activity Task, portanto deve ser definido como ligeiramente maior do que a duração máxima esperada para que a Activity seja concluída com sucesso. Um `start_to_close_timeout` de 5 segundos pode ser apropriado para uma Activity em um exemplo básico de Hello World, que deve retornar a saudação em uma fração de segundo:

```
# Start-to-Close Timeout should be set a little longer than the maximum
# length of time you expect for the Activity to complete successfully

result = await workflow.execute_activity_method(
    MyActivities.execute,
    input,
    start_to_close_timeout=timedelta(seconds=5),
)
```

No entanto, se você alterar o código da Activity para chamar um serviço remoto, processar um arquivo ou consultar um banco de dados, as execuções levarão consideravelmente mais tempo. Se você não atualizar o valor para algo maior do que o tempo de execução esperado, há uma grande chance de que as Activities expirem por timeout.

Embora especificar um `start_to_close_timeout` muito curto seja um problema, você também deve evitar um valor excessivamente longo. O `start_to_close_timeout` é como o Temporal detecta uma falha no Worker, portanto um valor muito alto desperdiça tempo ao atrasar a detecção e a recuperação, reduzindo o throughput. O `start_to_close_timeout` deve ser definido como ligeiramente maior do que a execução bem-sucedida mais lenta que você espera para a Activity.

Como o Temporal Lida com Falhas em Activities

Comportamento padrão

O comportamento padrão do Temporal é retentar automaticamente uma Activity, com um breve intervalo entre cada tentativa, até que ela seja concluída com sucesso ou cancelada. Isso significa que falhas intermitentes não exigem nenhuma ação da sua parte. Quando uma tentativa subsequente é bem-sucedida, seu código retoma como se a falha nunca tivesse ocorrido. No entanto, esse comportamento nem sempre é desejável, portanto o Temporal permite personalizá-lo por meio de uma Retry Policy customizada.

Ajustando o tempo e o número de tentativas

Quatro propriedades determinam o tempo e o número de retentativas:

Propriedade	Descrição	Valor padrão
`initial_interval`	Duração antes da primeira retentativa	1 segundo
`backoff_coefficient`	Multiplicador usado para retentativas subsequentes	2.0
`maximum_interval`	Duração máxima entre retentativas	100 * `initial_interval`
`maximum_attempts`	Número máximo de retentativas antes de desistir	0 (ilimitado)

`initial_interval` define quanto tempo após a falha inicial a primeira retentativa ocorrerá. Por padrão, esse valor é de um segundo.

`backoff_coefficient` é um multiplicador aplicado ao valor de `initial_interval` para calcular o intervalo entre cada tentativa subsequente. Usando os valores padrão de ambas as propriedades, haverá uma retentativa após 1 segundo, outra após 2 segundos, depois 4 segundos, 8 segundos e assim por diante.

`maximum_interval` impõe um limite a esse intervalo. Por padrão, é 100 vezes o `initial_interval`, o que significa que os intervalos continuariam aumentando conforme descrito, mas nunca ultrapassariam 100 segundos.

`maximum_attempts` especifica o número máximo de retentativas permitidas antes de marcar a Activity como falha, momento em que o Workflow pode tratar a falha de acordo com sua lógica de negócio.

Exemplo de Retry Policy em Activity

A imagem abaixo mostra um exemplo anotado de uma `RetryPolicy`. Há três etapas para usar uma `RetryPolicy` customizada para controlar como falhas em Activities são tratadas no seu Workflow:

1. Importar o pacote `workflow`
2. Especificar valores para uma ou mais propriedades, como `initial_interval` ou `backoff_coefficient`, descritas na página anterior
3. Associar sua policy ao parâmetro `retry_policy` usado com sua Activity

Activity Retry Policy Example

```python
from datetime import timedelta
from temporalio import workflow
from temporalio.common import RetryPolicy

# Import Activity, passing it through the sandbox without reloading the module
with workflow.unsafe.imports_passed_through():
    from translate import TranslateActivities


@workflow.defn
class GreetSomeone:
    @workflow.run
    async def run(self, name: str) -> str:
        retry_policy = RetryPolicy(
            initial_interval=timedelta(seconds=15), # first retry will occur after 15 seconds
            backoff_coefficient=2.0,                # double the delay after each retry
            maximum_interval=timedelta(seconds=160),# up to a maximum of 60 seconds
            maximum_attempts=100,                   # fail the Activity after 100 attempts
        )
        greeting = await workflow.execute_activity(
            TranslateActivities.greet_in_spanish,
            name,
            start_to_close_timeout=timedelta(seconds=5),
            retry_policy=retry_policy,
        )

        return f"{greeting}"
```

1 Import this package from the SDK

2 Specify your policy values

3 Associate the policy with the Activity options

Exercício Prático #3: Farewell Workflow

Durante este exercício, você irá:

- Criar uma Activity Definition que realiza chamadas a um microsserviço
- Registrar a Activity no Worker
- Modificar um Workflow existente para incluir a nova Activity
- Executar o Workflow e observar o resultado

Consulte o arquivo `README.md` no ambiente de exercícios para detalhes sobre este exercício. Você encontrará o código no diretório `exercises/farewell-workflow`. Lembre-se de fazer suas alterações no subdiretório `practice`.

Os comentários `TODO` indicarão onde as alterações são necessárias. Se precisar de uma dica ou quiser comparar suas mudanças, consulte a versão completa no subdiretório `solution`.

Sobre Este Exemplo

No exercício anterior, você executou um Workflow que incluía duas Activities, ambas fazendo chamadas a um microsserviço que fornecia mensagens personalizadas em espanhol. Esse exercício demonstra muitos dos conceitos-chave aprendidos neste curso. Embora você já tenha experiência prática com o desenvolvimento e execução de aplicações na Temporal Platform, você obterá uma compreensão mais profunda de como o Temporal funciona ao analisar o que acontece durante uma Workflow Execution.

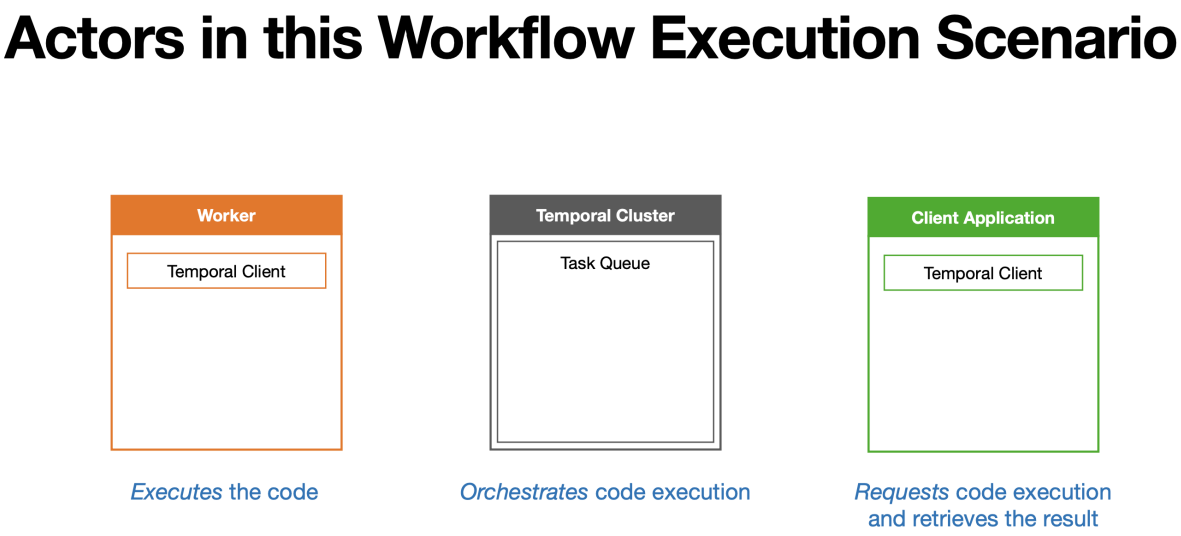
Os atores do cenário

O exemplo inclui três atores principais:

O **Worker** executa o código do Workflow e das Activities, e usa um Client para se comunicar com o Cluster.

O **Temporal Cluster** orquestra a execução desse código coordenando com o Worker por meio de uma Task Queue compartilhada.

O **Client Application** inicia o Workflow e solicita seu resultado ao Temporal Cluster, usando um Client para isso.



Atribuição de trabalho

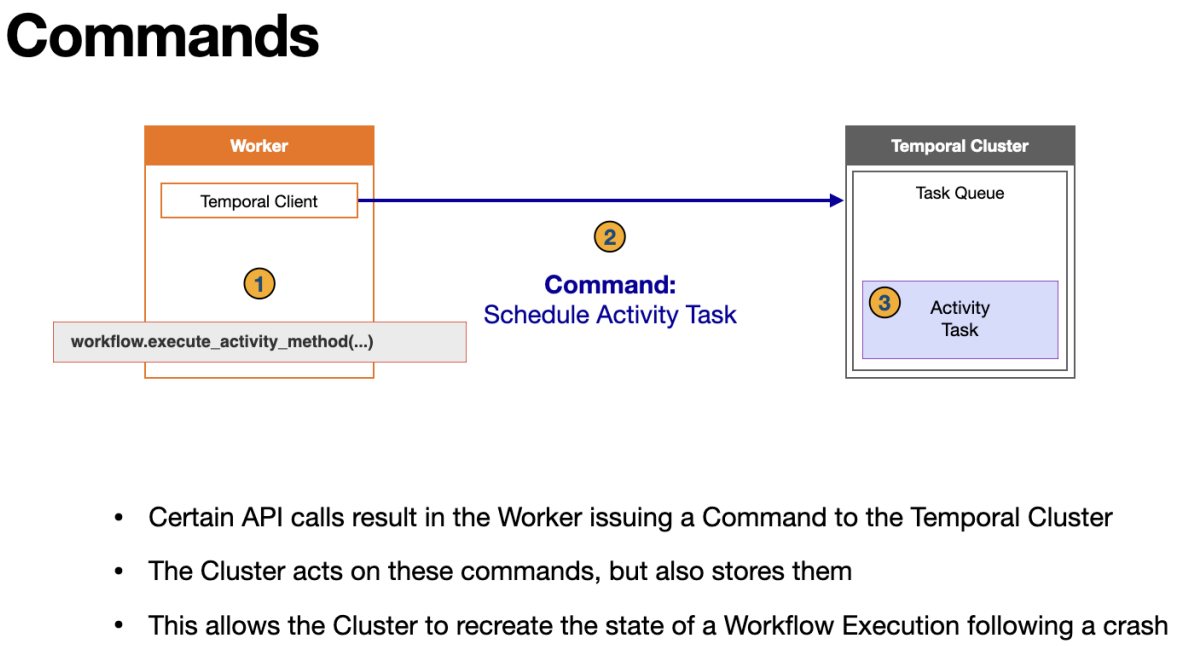
A atribuição de trabalho é indireta. O Temporal Cluster não atribui tarefas diretamente a um Worker e, na verdade, não mantém uma lista de Workers. Em vez disso, os Workers fazem polling contínuo na Task Queue do Temporal Cluster quando têm slots disponíveis para processar tarefas. Um dos benefícios dessa abordagem é que as tarefas permanecem na fila se não houver Workers suficientes, o que significa que você pode aumentar o throughput e a escalabilidade adicionando mais Workers.

Aplicações Temporal em produção normalmente possuem múltiplos Workers; no entanto, este exemplo usa um único Worker por simplicidade.

Commands

Outro conceito importante é o papel dos Commands. Quando o Worker encontra certas chamadas de API durante uma Workflow Execution, como uma chamada à função `execute_activity_method`, ele envia um Command ao Temporal Cluster. O Cluster age

sobre esses Commands, por exemplo criando uma Activity Task, e também os armazena em caso de falha.



Se o Worker travar, o Temporal Service trabalha com os Workers restantes para recriar o estado do Workflow imediatamente antes da falha e retoma o progresso a partir desse ponto. Isso permite que você, como desenvolvedor, escreva código como se esse tipo de falha nem fosse uma possibilidade.

O código do exemplo

A aplicação define duas Activities: `greet_in_spanish` e `farewell_in_spanish`:

```
import urllib.parse
from temporalio import activity

class TranslateActivities:
    def __init__(self, session):
        self.session = session

    @activity.defn
    async def greet_in_spanish(self, name: str) -> str:
        greeting = await self.call_service("get-spanish-greeting", name)
        return greeting

    @activity.defn
    async def farewell_in_spanish(self, name: str) -> str:
        farewell = await self.call_service("get-spanish-farewell", name)
        return farewell

    async def call_service(self, stem: str, name: str) -> str:
```

```
        base = f"http://localhost:9999/{stem}"
        url = f"{base}?name={urllib.parse.quote(name)}"

        async with self.session.get(url) as response:
            response.raise_for_status()
            return await response.text()
```

O Workflow Definition executa essas duas Activities e retorna uma string composta por seus resultados:

```
from datetime import timedelta
from temporalio import workflow

with workflow.unsafe.imports_passed_through():
    from translate import TranslateActivities


@workflow.defn
class GreetSomeone:
    @workflow.run
    async def run(self, name: str) -> str:
        greeting = await workflow.execute_activity_method(
            TranslateActivities.greet_in_spanish,
            name,
            start_to_close_timeout=timedelta(seconds=5),
        )

        farewell = await workflow.execute_activity_method(
            TranslateActivities.greet_in_spanish,
            name,
            start_to_close_timeout=timedelta(seconds=5),
        )

        return f"{greeting}\n{farewell}"
```

O código de inicialização do Worker registra os Workflow e Activity Definitions:

```
import asyncio
import aiohttp

from temporalio.client import Client
from temporalio.worker import Worker

from translate import TranslateActivities
from greeting import GreetSomeone


async def main():
```

```
    client = await Client.connect("localhost:7233", namespace="default")

    async with aiohttp.ClientSession() as session:
        activities = TranslateActivities(session)

        worker = Worker(
            client,
            task_queue="greeting-tasks",
            workflows=[GreetSomeone],
            activities=[activities.greet_in_spanish,
activities.farewell_in_spanish],
        )
        print("Starting the worker....")
        await worker.run()


if __name__ == "__main__":
    asyncio.run(main())
```

Neste curso, você viu como as partes de uma aplicação Temporal, o Worker, o Temporal Cluster e o Client Application, trabalham juntos durante uma Workflow Execution. No próximo vídeo, você verá como todas as partes se integram por meio de um walkthrough do código.

Code Walkthrough

Transcrição do Vídeo

Como você aprendeu, o Worker executa o código do Workflow e das Activities, portanto uma Workflow Execution não pode avançar a menos que pelo menos um Worker esteja em execução. Ao iniciar o Worker, um novo processo é criado. Como este exemplo é escrito em Python, você pode definir uma função `main` e usá-la para iniciar a execução.

Essa função primeiro cria um Temporal Client. Em seguida, cria uma nova entidade Worker com esse cliente, o nome da Task Queue e os Workflows e Activities registrados no Worker.

Ao executar a entidade Worker, uma conexão de longa duração com o Temporal Cluster é aberta, usada para fazer polling contínuo por novas tarefas. Embora o Worker esteja em execução, o Workflow ainda não está, portanto a Task Queue está vazia e o Worker não tem nada a fazer.

Uma forma de iniciar o Workflow é com a ferramenta de linha de comando `temporal`. Este exemplo especifica o nome da Task Queue do Worker, um Workflow ID definido pelo usuário, o Workflow Type e os dados de entrada. Uma alternativa é iniciá-lo a partir do

código da sua própria aplicação, usando um Temporal Client para chamar o método `start_workflow` com a entrada desejada.

Independentemente de como o Workflow é iniciado, o comportamento será o mesmo: o Temporal Cluster registra um novo Event no Event History desta Workflow Execution. `WorkflowExecutionStarted` é sempre o primeiro Event.

O Temporal Cluster adiciona um Workflow Task à Task Queue e registra outro evento, `WorkflowTaskScheduled`, no Event History. O nome segue um padrão: quando uma nova tarefa é adicionada à fila, o nome termina com "Scheduled".

Como o processo do Worker tem capacidade para processar trabalho, ele aceita essa nova tarefa. Isso resulta em um novo Event cujo nome também segue um padrão: quando um Worker retira uma tarefa da fila, o Cluster gera um evento cujo nome termina com "Started".

O processo do Worker inicia o Workflow Task invocando o método do Workflow Definition. No Python SDK, as opções de Activity como `start_to_close_timeout` são passadas como argumentos nomeados ao executar a Activity.

O código do Workflow declara uma variável que receberá a saída da primeira Activity e então solicita a execução de `greet_in_spanish`. Como `execute_activity_method` aguarda a conclusão da Activity antes de continuar, o resultado é armazenado em `greeting` e fica imediatamente disponível.

Algumas coisas importantes acontecem como resultado da chamada a `execute_activity_method`. O Worker não pode avançar no Workflow até que a Activity Execution seja concluída, então ele notifica o Cluster de que o Workflow Task atual está completo. Em resposta, o Cluster adiciona um novo Event ao histórico. O Worker também envia um Command ao Cluster solicitando que agende uma Activity Task.

O Temporal Cluster cria uma Activity Task e a adiciona à Task Queue, gerando um novo Event. Como o processo do Worker tem capacidade para trabalho adicional, ele aceita a Activity Task e invoca o método correspondente à Activity Definition de `greet_in_spanish`, que por sua vez chama o microsserviço.

A requisição é bem-sucedida e o serviço responde com uma saudação personalizada em espanhol. Quando o método da Activity retorna, o Worker notifica o Cluster de que a Activity Task está completa, gerando um novo Event. Em resposta, o Temporal Cluster enfileira um novo Workflow Task e registra outro Event.

O Worker continua de onde parou, executando a próxima instrução no Workflow Definition. É hora de executar a segunda Activity, então o Worker notifica o Temporal Cluster de que o Workflow Task atual está completo e envia um Command para agendar uma Activity Task.

Vamos pausar para analisar um cenário de falha. O que acontece se o Worker travar por falta de memória, por exemplo? Você pode se recuperar disso reiniciando o Worker ou iniciando um novo em outra máquina. Em ambos os casos, o Temporal recriará

automaticamente o estado do Workflow até o ponto da falha, e o progresso continuará a partir daí, como se o Worker nunca tivesse travado.

Activities que foram concluídas com sucesso antes da falha não serão executadas novamente; o Temporal reutiliza os valores retornados por suas execuções anteriores.

Quando o Worker aceita a Activity Task, o Temporal Cluster adiciona `ActivityTaskStarted` ao Event History e o Worker invoca o método da segunda Activity, que usa o método utilitário para chamar o microsserviço.

Mas e se o microsserviço tiver ficado offline antes da requisição? Nesse caso, a requisição falharia, fazendo com que o método da Activity lançasse uma exceção. O comportamento padrão do Temporal é retentar automaticamente uma Activity que falhou, com um breve intervalo, até que ela seja concluída ou cancelada. Você pode personalizar esse comportamento com uma Retry Policy.

Por meio de uma retentativa, o Worker invoca o método da Activity novamente, que por sua vez chama o microsserviço. Assumindo que a falha foi intermitente, a requisição feita durante a retentativa é bem-sucedida e o serviço responde com a mensagem de despedida solicitada.

Quando o método retorna, o Worker notifica o Temporal Cluster de que a Activity Task está completa. Como ainda há linhas do código do Workflow a serem executadas, o Temporal Cluster adiciona um novo Workflow Task à fila. O Worker continua de onde parou, executando as instruções restantes no Workflow Definition.

Assim que o método retorna, o Workflow Task está completo. Como o método do Workflow retornou, a Workflow Execution está concluída e o Cluster adiciona o evento final ao histórico.

O Worker continua fazendo polling por novas tarefas, mas não há mais trabalho relacionado a esta Workflow Execution. O Client Application, que estava aguardando o resultado da Workflow Execution, receberá esse valor. O Cluster fornece o resultado à aplicação, que pode processá-lo como desejar.

Exercício Prático #4: Finale Workflow

Durante este exercício, você irá:

- Executar um Workflow escrito em Python que usa uma Activity escrita em Java
- Ver um exemplo de como o Temporal pode ser usado para tarefas de processamento de arquivos
- Obter a comprovação de que você concluiu este curso com sucesso

Consulte o arquivo `README.md` no ambiente de exercícios para detalhes sobre este exercício. Você encontrará o código no diretório `exercises/finale-workflow`. Como o exercício envolve apenas a execução do código, não a escrita, não há subdiretórios `practice` ou `solution`.

Conclusão

Pontos essenciais

Aqui está um resumo dos conceitos mais importantes aprendidos durante o curso:

- O Temporal garante a execução durável das suas aplicações, e os Workflows são definidos por meio de código usando um Temporal SDK.
- Os Temporal Clusters orquestram a execução do código, enquanto os Workers são responsáveis por executá-lo de fato. O Frontend Service aceita requisições de clientes e as encaminha para o serviço de backend apropriado, usando gRPC com suporte a TLS.
- O Temporal Cluster mantém Task Queues criadas dinamicamente. Os Workers fazem polling contínuo em uma Task Queue e aceitam tarefas se tiverem capacidade disponível. Você pode aumentar a escalabilidade adicionando mais Workers e deve reiniciá-los após implantar uma alteração no código.
- Existem múltiplas formas de implantar um Temporal Cluster self-hosted. O Temporal Cloud, com precificação baseada em consumo, é uma alternativa conveniente, e a migração para ou a partir dele requer poucas alterações no código da aplicação.
- Namespaces são usados para isolamento dentro de um cluster, permitindo separar logicamente aplicações por status ou equipe responsável.
- No Python SDK, um Temporal Workflow é definido por meio de uma classe, enquanto as Activities são definidas por meio de funções.
- Activities encapsulam código não confiável ou não determinístico, como invocações de LLMs, chamadas de API, consultas a bancos de dados e operações de arquivo. Elas são automaticamente retentadas em caso de falha, e o comportamento de retentativa pode ser personalizado por meio de uma Retry Policy.
- Como o trabalho não determinístico é realizado por Activities, o Temporal pode ser usado para construir aplicações de IA duráveis sem violar o determinismo do Workflow.
- A Web UI é uma ferramenta poderosa para obter visibilidade sobre sua aplicação, exibindo Workflow Executions atuais e recentes, além de entradas, saídas e Event History.

Palavras finais

Durante este curso, você desenvolveu e executou aplicações na Temporal Platform. O curso começou explorando casos de uso reais para Workflows, seguido pelos fundamentos da arquitetura do Temporal. Você atualizou o código de inicialização do Worker, especificou a

Task Queue que o Worker monitoraria no Temporal Cluster, iniciou o Worker e usou o `temporal` para iniciar um Workflow.

Em seguida, usou a Temporal Web UI para localizar informações sobre aquela Workflow Execution, incluindo o Workflow Type, o nome da Task Queue, os horários de início e encerramento e o namespace utilizado. Posteriormente, adicionou uma nova Activity a um Workflow, registrou essa Activity no Worker, atualizou o Workflow Definition para solicitar a execução dessa Activity e usou código da aplicação para iniciar o Workflow. As Activities chamavam um microsserviço, e você experimentou como uma Retry Policy do Temporal garante que suas Activities sejam automaticamente retentadas em caso de falha.

Por fim, você viu o que acontece nos bastidores durante a execução do Workflow, incluindo a interação entre o Worker e o Temporal Cluster, os eventos gerados durante a execução e como a execução do seu código permanece durável mesmo diante de diferentes cenários de falha. No exercício final, você aproveitou o suporte poliglota da Temporal Platform executando um Workflow escrito em Python que usou uma Activity escrita em Java para gerar um certificado de conclusão do curso.

Obrigado por participar do Temporal 101!

Para mais informações

Estes recursos irão ajudá-lo no desenvolvimento das suas próprias aplicações Temporal:

- Documentação do Temporal
- Fórum da Comunidade Temporal
- Slack da Comunidade Temporal
- Repositório `python-samples` do Temporal no GitHub
- Canal do Temporal no YouTube
- Eventos da Comunidade Temporal